

The 10 Essential Features for the Future of C++ Libraries

Mateusz Pusz

16

The ~~10~~ Essential Features for the Future of C++ Libraries

Mateusz Pusz

C++ Evolution

C++ Evolution

- C++11 - Cornerstone of Modern C++

- Move Semantics, **constexpr**, lambda expressions, variable templates, memory model and threads, range-for, chrono, ...

C++ Evolution

- C++11 - Cornerstone of Modern C++

- Move Semantics, **constexpr**, lambda expressions, variable templates, memory model and threads, range-for, chrono, ...

- C++20 - HUGE!!!

- concepts, ranges, coroutines, class NTTPs, **constexpr** all the things, **constexpr**, **<=>**, **format**, **counting_semaphore/latch/barrier**, modules 🤖, ...

C++ Evolution

- C++11 - Cornerstone of Modern C++

- Move Semantics, **constexpr**, lambda expressions, variable templates, memory model and threads, range-for, chrono, ...

- C++20 - HUGE!!!

- concepts, ranges, coroutines, class NTTPs, **constexpr** all the things, **constexpr**, **<=>**, **format**, **counting_semaphore/latch/barrier**, modules 🤖, ...

- C++26 - AWESOME!!!

- contracts, **std::execution**, even more **constexpr**, relocability, RCU, hazard pointers, SIMD, linalg, reflection (?), ...

C++ Evolution

- C++11 - Cornerstone of Modern C++

- Move Semantics, **constexpr**, lambda expressions, variable templates, memory model and threads, range-for, chrono, ...

- C++20 - HUGE!!!

- concepts, ranges, coroutines, class NTTPs, **constexpr** all the things, **constexpr**, **<=>**, **format**, **counting_semaphore/latch/barrier**, modules 🤔, ...

- C++26 - AWESOME!!!

- contracts, **std::execution**, even more **constexpr**, relocability, RCU, hazard pointers, SIMD, linalg, reflection (?), ...

I will...

- 1 Describe problems that library developers and their users face today

I will...

1 Describe problems that library developers and their users face today

- Improving interfaces
- Enabling compile-time code debugging
- Better compile-time error messages
- Improving safety
- A need for additional text utilities
- Improving distribution and consumption of libraries

I will...

1 Describe problems that library developers and their users face today

- Improving interfaces
- Enabling compile-time code debugging
- Better compile-time error messages
- Improving safety
- A need for additional text utilities
- Improving distribution and consumption of libraries

The feature selection is subjective and far from being complete...

I will...

- 1 Describe problems that library developers and their users face today
- 2 Present how those problems can be addressed with future C++ language and library features

I will...

- 1 Describe problems that library developers and their users face today
- 2 Present how those problems can be addressed with future C++ language and library features
 - Already **accepted for C++26**
 - *"C++26"* annotation next to a paper number in a title

I will...

- 1 Describe problems that library developers and their users face today
- 2 Present how those problems can be addressed with future C++ language and library features
 - Already **accepted for C++26**
 - *"C++26"* annotation next to a paper number in a title
 - **In the WG21 pipeline** that did not get to C++26
 - just a paper number in a title

I will...

- 1 Describe problems that library developers and their users face today
- 2 Present how those problems can be addressed with future C++ language and library features
 - Already **accepted for C++26**
 - "**C++26**" annotation next to a paper number in a title
 - **In the WG21 pipeline** that did not get to C++26
 - just a paper number in a title
 - **Abandoned papers** that did not have a revision for a longer time
 - "**Abandoned**" annotation next to a paper number in a title
 - *Contributors are welcome to help progress them*

I will...

- 1 Describe problems that library developers and their users face today
- 2 Present how those problems can be addressed with future C++ language and library features
- 3 NOT discuss big features in details (e.g., reflection, contracts)

I will...

- 1 Describe problems that library developers and their users face today
- 2 Present how those problems can be addressed with future C++ language and library features
- 3 NOT discuss big features in details (e.g., reflection, contracts)
 - Not enough time
 - Many great talks about them already available

I will...

- 1 Describe problems that library developers and their users face today
- 2 Present how those problems can be addressed with future C++ language and library features
- 3 NOT discuss big features in details (e.g., reflection, contracts)
- 4 Present concrete examples of how they solve common problems in Modern C++ libraries

Finding a paper

- Get paper
 - <https://wg21.link/nXXXX>
 - <https://wg21.link/pXXXXrX>
 - <https://wg21.link/pXXXX> - latest version

Finding a paper

- Get paper
 - <https://wg21.link/nXXXX>
 - <https://wg21.link/pXXXXrX>
 - <https://wg21.link/pXXXX> - latest version
- Get GitHub tracking issue
 - <https://wg21.link/pXXXX/github>
 - <https://wg21.link/pXXXX/status>

Finding a paper

- Get paper
 - <https://wg21.link/nXXXX>
 - <https://wg21.link/pXXXXrX>
 - <https://wg21.link/pXXXX> - latest version
- Get GitHub tracking issue
 - <https://wg21.link/pXXXX/github>
 - <https://wg21.link/pXXXX/status>
- Get working draft
 - <https://wg21.link/std{11,14,17,20,23}>
 - <https://wg21.link/std> - latest released version

Finding a paper

- Get paper
 - <https://wg21.link/nXXXX>
 - <https://wg21.link/pXXXXrX>
 - <https://wg21.link/pXXXX> - latest version
- Get GitHub tracking issue
 - <https://wg21.link/pXXXX/github>
 - <https://wg21.link/pXXXX/status>
- Get working draft
 - <https://wg21.link/std{11,14,17,20,23}>
 - <https://wg21.link/std> - latest released version
- Usage info
 - <https://wg21.link>

IMPROVING INTERFACES

How do you feel about such an interface?

```
void* foo(void* t) { /* ... */ }
```

How do you feel about such an interface?

```
void* foo(void* t) { /* ... */ }
```

```
auto foo = [](auto&& t) { /* ... */ };
```


How do you feel about such an interface?

```
void* foo(void* t) { /* ... */ }
```

```
auto foo = [](auto&& t) { /* ... */ };
```

```
template<typename T> auto foo(T&& t) { /* ... */ }
```

How do you feel about such an interface?

```
void* foo(void* t) { /* ... */ }
```

```
auto foo = [](auto&& t) { /* ... */ };
```

```
auto foo(auto&& t) { /* ... */ }
```

How do you feel about such an interface?

```
void* foo(void* t) { /* ... */ }
```

```
auto foo = [](auto&& t) { /* ... */ };
```

```
auto foo(auto&& t) { /* ... */ }
```

```
template<typename T> class foo { /* ... */ };
```

How do you feel about such an interface?

```
void* foo(void* t) { /* ... */ }
```

```
auto foo = [](auto&& t) { /* ... */ };
```

```
auto foo(auto&& t) { /* ... */ }
```

```
template<typename T> class foo { /* ... */ };
```

Unconstrained template parameters are the **void*** of C++

Legacy template code

```
template<auto R, typename Rep = double>
class quantity {
public:
    template<auto R2, typename Rep2>
    [[nodiscard]] friend constexpr auto operator/(const quantity& lhs,
                                                    const quantity<R2, Rep2>& rhs);
    // ...
};
```

Legacy template code

```
template<auto R, typename Rep = double>
class quantity {
public:
    template<auto R2, typename Rep2>
    [[nodiscard]] friend constexpr auto operator/(const quantity& lhs,
                                                    const quantity<R2, Rep2>& rhs);

    // ...
};
```

- What should I pass as R and Rep?

Legacy template code

```
template<auto R, typename Rep = double>
class quantity {
public:
    template<auto R2, typename Rep2>
    [[nodiscard]] friend constexpr auto operator/(const quantity& lhs,
                                                    const quantity<R2, Rep2>& rhs);
    // ...
};
```

- What should I pass as **R** and **Rep**?
- What if **Rep** can't be divided with **Rep2**?

Legacy template code

```
template<auto R, typename Rep = double>
class quantity {
public:
    template<auto R2, typename Rep2>
    [[nodiscard]] friend constexpr auto operator/(const quantity& lhs,
                                                    const quantity<R2, Rep2>& rhs);
    // ...
};
```

- What should I pass as **R** and **Rep**?
- What if **Rep** can't be divided with **Rep2**?
- What is returned from a function?

Legacy template code

```
template<auto R, typename Rep = double>
class quantity {
public:
    template<auto R2, typename Rep2>
    [[nodiscard]] friend constexpr auto operator/(const quantity& lhs,
                                                    const quantity<R2, Rep2>& rhs);
    // ...
};
```

- What should I pass as R and Rep?
- What if Rep can't be divided with Rep2?
- What is returned from a function?

If I will manage to crash the compilation of your code, then it is your bug!

Legacy template code

```
template<auto R, typename Rep = double>
class quantity {
public:
    template<auto R2, typename Rep2>
    [[nodiscard]] friend constexpr auto operator/(const quantity& lhs,
                                                    const quantity<R2, Rep2>& rhs);
    // ...
};
```

Can you spot a bug?

Legacy template code

```
template<auto R, typename Rep = double>
class quantity {
public:
    template<auto R2, typename Rep2>
    [[nodiscard]] friend constexpr auto operator/(const quantity& lhs,
                                                    const quantity<R2, Rep2>& rhs);

    // ...
};
```

DEPENDENT

- Parameter/Argument adjustments
- Consideration of alternatives
- No additional conversions

NOT DEPENDENT

- Trivial Conversions
- Promotions
- Standard Conversions
- User-defined Conversions

Concepts are great!

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<std::derived_from<quantity> Q, auto R2, typename Rep2>
        requires detail::InvocableQuantities<std::divides<>, quantity, quantity<R2, Rep2>>
        [[nodiscard]] friend constexpr Quantity auto operator/(const Q& lhs,
                                                                const quantity<R2, Rep2>& rhs);
    // ...
};
```

Concepts are great!

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<std::derived_from<quantity> Q, auto R2, typename Rep2>
        requires detail::InvocableQuantities<std::divides<>, quantity, quantity<R2, Rep2>>
        [[nodiscard]] friend constexpr Quantity auto operator/(const Q& lhs,
                                                                const quantity<R2, Rep2>& rhs);
    // ...
};
```

Challenge every "naked" **typename** and **auto** in your code.

It's only a half of the story 🤔

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<std::derived_from<quantity> Q, auto R2, typename Rep2>
        requires detail::InvocableQuantities<std::divides<>, quantity, quantity<R2, Rep2>>
        [[nodiscard]] friend constexpr Quantity auto operator/(const Q& lhs,
                                                                const quantity<R2, Rep2>& rhs)
        {
            MP_UNITS_EXPECTS_DEBUG(is_neq_zero(rhs));
            return ::mp_units::quantity{lhs.numerical_value_ref_in(unit) /
                                         rhs.numerical_value_ref_in(rhs.unit),
                                         R / R2};
        }
    //...
};
```

It's only a half of the story 🤔

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<std::derived_from<quantity> Q, auto R2, typename Rep2>
        requires detail::InvocableQuantities<std::divides<>, quantity, quantity<R2, Rep2>>
        [[nodiscard]] friend constexpr Quantity auto operator/(const Q& lhs,
                                                                const quantity<R2, Rep2>& rhs)
        {
            MP_UNITS_EXPECTS_DEBUG(is_neq_zero(rhs));
            return ::mp_units::quantity{lhs.numerical_value_ref_in(unit) /
                                         rhs.numerical_value_ref_in(rhs.unit),
                                         R / R2};
        }
    // ...
};
```

- *Not visible in function declarations* and automatically generated *API reference* documentation

It's only a half of the story 🙄

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<std::derived_from<quantity> Q, auto R2, typename Rep2>
        requires detail::InvocableQuantities<std::divides<>, quantity, quantity<R2, Rep2>>
        [[nodiscard]] friend constexpr Quantity auto operator/(const Q& lhs,
                                                                const quantity<R2, Rep2>& rhs)
        {
            MP_UNITS_EXPECTS_DEBUG(is_neq_zero(rhs));
            return ::mp_units::quantity{lhs.numerical_value_ref_in(unit) /
                                         rhs.numerical_value_ref_in(rhs.unit),
                                         R / R2};
        }
    // ...
};
```

- *Not visible in function declarations* and automatically generated *API reference* documentation
- Often *checked only in Debug* builds

P2900 Contracts for C++ (C++26) 🎉

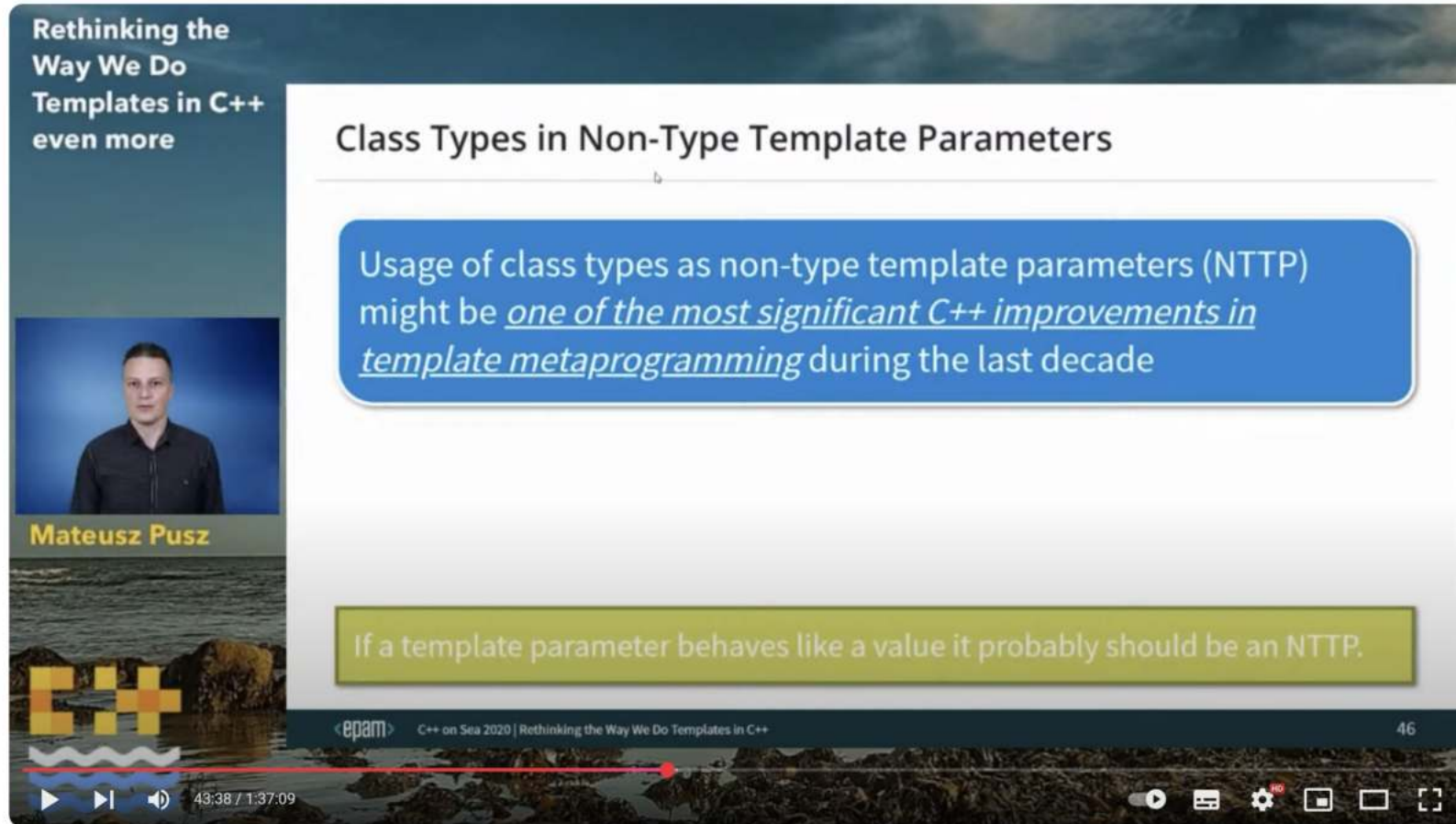
```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<std::derived_from<quantity> Q, auto R2, typename Rep2>
        requires detail::InvocableQuantities<std::divides<>, quantity, quantity<R2, Rep2>>
        [[nodiscard]] friend constexpr Quantity auto operator/(const Q& lhs,
                                                                const quantity<R2, Rep2>& rhs)
        pre (is_neq_zero(rhs))
        {
            return ::mp_units::quantity{lhs.numerical_value_ref_in(unit) /
                                         rhs.numerical_value_ref_in(rhs.unit),
                                         R / R2};
        }
    //...
};
```

P2900 Contracts for C++ (C++26) 🎉

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<std::derived_from<quantity> Q, auto R2, typename Rep2>
        requires detail::InvocableQuantities<std::divides<>, quantity, quantity<R2, Rep2>>
        [[nodiscard]] friend constexpr Quantity auto operator/(const Q& lhs,
                                                                const quantity<R2, Rep2>& rhs)
        pre (is_neq_zero(rhs))
        {
            return ::mp_units::quantity{lhs.numerical_value_ref_in(unit) /
                                         rhs.numerical_value_ref_in(rhs.unit),
                                         R / R2};
        }
    //...
};
```

Being able to express compile-time and runtime contracts in declarations of our interfaces is AWESOME!

Supercharging Non-Type Template Parameters



The video player shows a presentation slide titled "Rethinking the Way We Do Templates in C++ even more" by Mateusz Pusz. The slide content is as follows:

Class Types in Non-Type Template Parameters

Usage of class types as non-type template parameters (NTTP) might be one of the most significant C++ improvements in template metaprogramming during the last decade

If a template parameter behaves like a value it probably should be an NTTP.

Video player controls at the bottom show the video is at 43:38 / 1:37:09. The Eppam logo and "C++ on Sea 2020 | Rethinking the Way We Do Templates in C++" are visible in the bottom left of the video frame.

Rethinking the Way We Do Templates in C++ even more - Mateusz Pusz [C++ on Sea 2020]

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;  
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;  
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
```

```
inline constexpr struct length final : quantity_spec<dim_length> {} length;  
inline constexpr struct time final : quantity_spec<dim_time> {} time;  
inline constexpr struct speed final : quantity_spec<length / time> {} speed;
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;  
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
```

```
inline constexpr struct length final : quantity_spec<dim_length> {} length;  
inline constexpr struct time final : quantity_spec<dim_time> {} time;  
inline constexpr struct speed final : quantity_spec<length / time> {} speed;
```

```
inline constexpr struct metre final : named_unit<"m", kind_of<isq::length>> {} metre;  
inline constexpr struct second final : named_unit<"s", kind_of<isq::time>> {} second;  
inline constexpr struct hour final : named_unit<"h", mag<60> * minute> {} hour;
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;  
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
```

```
inline constexpr struct length final : quantity_spec<dim_length> {} length;  
inline constexpr struct time final : quantity_spec<dim_time> {} time;  
inline constexpr struct speed final : quantity_spec<length / time> {} speed;
```

```
inline constexpr struct metre final : named_unit<"m", kind_of<isq::length>> {} metre;  
inline constexpr struct second final : named_unit<"s", kind_of<isq::time>> {} second;  
inline constexpr struct hour final : named_unit<"h", mag<60> * minute> {} hour;
```

```
template<PrefixableUnit U> struct kilo_ final : prefixed_unit<"k", mag_power<10, 3>, U{}> {};  
template<PrefixableUnit auto U> constexpr kilo_<decltype(U)> kilo;
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;  
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
```

```
inline constexpr struct length final : quantity_spec<dim_length> {} length;  
inline constexpr struct time final : quantity_spec<dim_time> {} time;  
inline constexpr struct speed final : quantity_spec<length / time> {} speed;
```

```
inline constexpr struct metre final : named_unit<"m", kind_of<isq::length>> {} metre;  
inline constexpr struct second final : named_unit<"s", kind_of<isq::time>> {} second;  
inline constexpr struct hour final : named_unit<"h", mag<60> * minute> {} hour;
```

```
template<PrefixableUnit U> struct kilo_ final : prefixed_unit<"k", mag_power<10, 3>, U{}> {};  
template<PrefixableUnit auto U> constexpr kilo_<decltype(U)> kilo;
```

```
quantity<speed[kilo<metre>/hour]> v = 120 * kilo<metre> / (2 * hour);
```


Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;  
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
```

```
inline constexpr struct length final : quantity_spec<dim_length> {} length;  
inline constexpr struct time final : quantity_spec<dim_time> {} time;  
inline constexpr struct speed final : quantity_spec<length / time> {} speed;
```

```
inline constexpr struct metre final : named_unit<"m", kind_of<isq::length>> {} metre;  
inline constexpr struct second final : named_unit<"s", kind_of<isq::time>> {} second;  
inline constexpr struct hour final : named_unit<"h", mag<60> * minute> {} hour;
```

```
template<PrefixableUnit U> struct kilo_ final : prefixed_unit<"k", mag_power<10, 3>, U{}> {};  
template<PrefixableUnit auto U> constexpr kilo_<decltype(U)> kilo;
```

```
quantity<speed[kilo<metre>/hour]> v = 120 * kilo<metre> / (2 * hour);
```

Non-type template parameters (NTTP) (C++20)

One of the following (optionally cv-qualified) types

- a **structural type**
- a type that *contains a placeholder* (**auto**) type
- a placeholder for a *deduced class type* (CTAD)

Non-type template parameters (NTTP) (C++20)

One of the following (optionally cv-qualified) types

- a **structural type**
- a type that *contains a placeholder* (**auto**) type
- a placeholder for a *deduced class type* (CTAD)

Structural types

- a scalar type with a constant destruction
- an lvalue reference type
- a literal class where all base classes and non-static data members are public and non-mutable structural types

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_thermodynamic_temperature final :  
    base_dimension<symbol_text{u8"Θ", "0"}> {} dim_thermodynamic_temperature;  
  
inline constexpr struct thermodynamic_temperature final :  
    quantity_spec<dim_thermodynamic_temperature> {} thermodynamic_temperature;  
  
inline constexpr struct absolute_zero final : absolute_point_origin<isq::thermodynamic_temperature> {} absolute_zero;  
  
inline constexpr struct kelvin final : named_unit<"K", kind_of<isq::thermodynamic_temperature>, absolute_zero> {} kelvin;
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_thermodynamic_temperature final :  
    base_dimension<symbol_text{u8"Θ", "0"}> {} dim_thermodynamic_temperature;  
  
inline constexpr struct thermodynamic_temperature final :  
    quantity_spec<dim_thermodynamic_temperature> {} thermodynamic_temperature;  
  
inline constexpr struct absolute_zero final : absolute_point_origin<isq::thermodynamic_temperature> {} absolute_zero;  
  
inline constexpr struct kelvin final : named_unit<"K", kind_of<isq::thermodynamic_temperature>, absolute_zero> {} kelvin;  
  
constexpr quantity delta = 273'150 * milli<kelvin>;  
constexpr quantity_point point = absolute_zero + delta;
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_thermodynamic_temperature final :  
    base_dimension<symbol_text{u8"Θ", "0"}> {} dim_thermodynamic_temperature;  
  
inline constexpr struct thermodynamic_temperature final :  
    quantity_spec<dim_thermodynamic_temperature> {} thermodynamic_temperature;  
  
inline constexpr struct absolute_zero final : absolute_point_origin<isq::thermodynamic_temperature> {} absolute_zero;  
  
inline constexpr struct kelvin final : named_unit<"K", kind_of<isq::thermodynamic_temperature>, absolute_zero> {} kelvin;
```

```
constexpr quantity delta = 273'150 * milli<kelvin>;  
constexpr quantity_point point = absolute_zero + delta;
```

```
inline constexpr struct ice_point final : relative_point_origin<point> {} ice_point;  
  
inline constexpr struct degree_Celsius final : named_unit<symbol_text{u8"°C", "'C"}, kelvin, ice_point> {} degree_Celsius;
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_thermodynamic_temperature final :  
    base_dimension<symbol_text{u8"Θ", "0"}> {} dim_thermodynamic_temperature;  
  
inline constexpr struct thermodynamic_temperature final :  
    quantity_spec<dim_thermodynamic_temperature> {} thermodynamic_temperature;  
  
inline constexpr struct absolute_zero final : absolute_point_origin<isq::thermodynamic_temperature> {} absolute_zero;  
  
inline constexpr struct kelvin final : named_unit<"K", kind_of<isq::thermodynamic_temperature>, absolute_zero> {} kelvin;
```

```
constexpr quantity delta = 273'150 * milli<kelvin>;  
constexpr quantity_point point = absolute_zero + delta;
```

```
inline constexpr struct ice_point final : relative_point_origin<point> {} ice_point;  
  
inline constexpr struct degree_Celsius final : named_unit<symbol_text{u8"°C", "'C"}, kelvin, ice_point> {} degree_Celsius;
```

```
quantity_point room_temp(21 * degree_Celsius);
```

Supercharging Non-Type Template Parameters

```
inline constexpr struct dim_thermodynamic_temperature final :  
    base_dimension<symbol_text{u8"Θ", "0"}> {} dim_thermodynamic_temperature;  
  
inline constexpr struct thermodynamic_temperature final :  
    quantity_spec<dim_thermodynamic_temperature> {} thermodynamic_temperature;  
  
inline constexpr struct absolute_zero final : absolute_point_origin<isq::thermodynamic_temperature> {} absolute_zero;  
  
inline constexpr struct kelvin final : named_unit<"K", kind_of<isq::thermodynamic_temperature>, absolute_zero> {} kelvin;
```

```
constexpr quantity delta = 273'150 * milli<kelvin>;  
constexpr quantity_point point = absolute_zero + delta;
```

```
inline constexpr struct ice_point final : relative_point_origin<point> {} ice_point;  
  
inline constexpr struct degree_Celsius final : named_unit<symbol_text{u8"°C", "'C"}, kelvin, ice_point> {} degree_Celsius;
```

```
quantity_point room_temp(21 * degree_Celsius);
```


Supercharging Non-Type Template Parameters

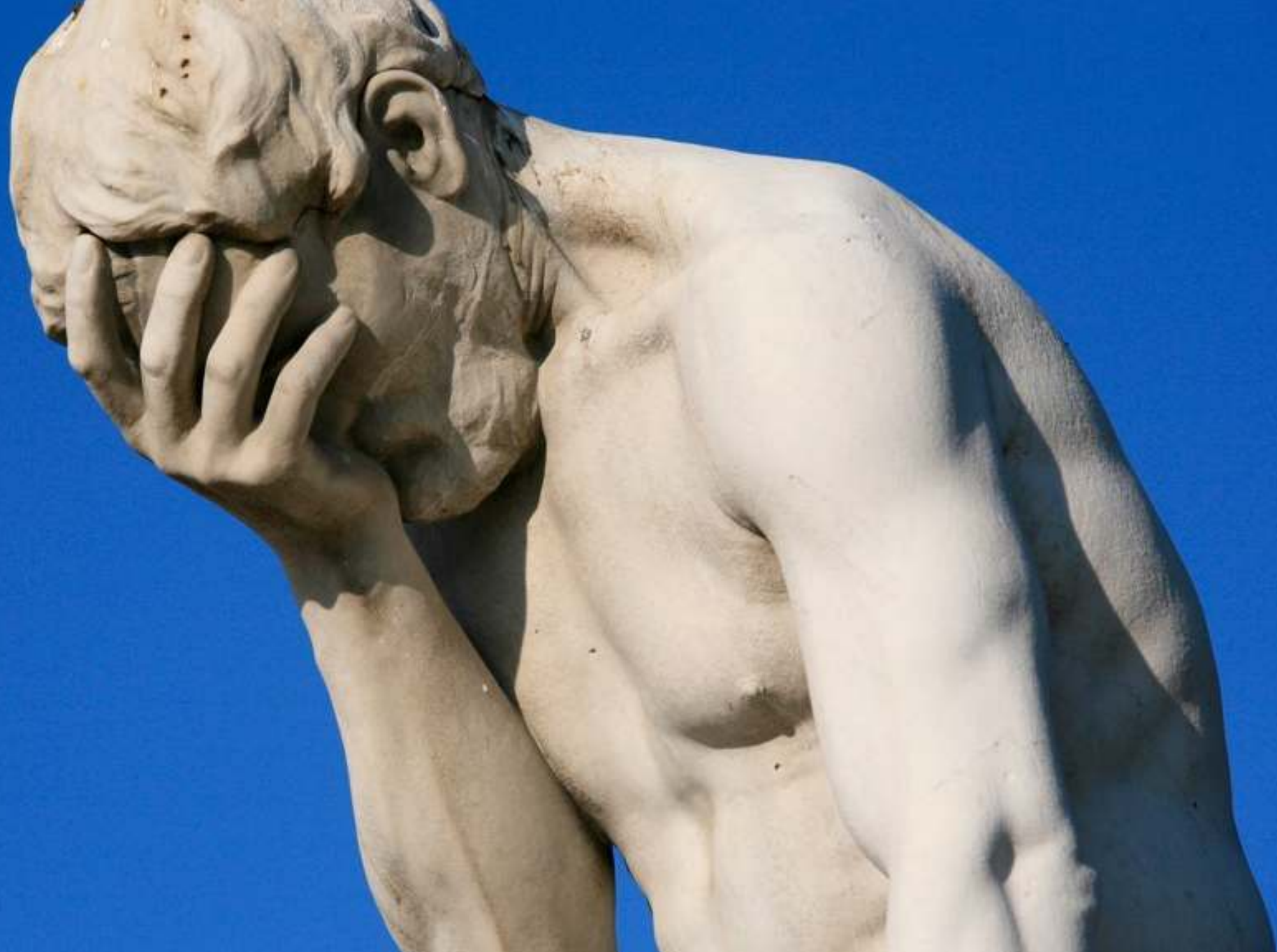
```
inline constexpr struct dim_thermodynamic_temperature final :  
    base_dimension<symbol_text{u8"Θ", "0"}> {} dim_thermodynamic_temperature;  
  
inline constexpr struct thermodynamic_temperature final :  
    quantity_spec<dim_thermodynamic_temperature> {} thermodynamic_temperature;  
  
inline constexpr struct absolute_zero final : absolute_point_origin<isq::thermodynamic_temperature> {} absolute_zero;  
  
inline constexpr struct kelvin final : named_unit<"K", kind_of<isq::thermodynamic_temperature>, absolute_zero> {} kelvin;
```

```
constexpr quantity delta = 273'150 * milli<kelvin>;  
constexpr quantity_point point = absolute_zero + delta;
```

```
inline constexpr struct ice_point final : relative_point_origin<point> {} ice_point;  
  
inline constexpr struct degree_Celsius final : named_unit<symbol_text{u8"°C", "'C"}, kelvin, ice_point> {} degree_Celsius;
```

```
quantity_point room_temp(21 * degree_Celsius);
```

```
room_temp.quantity_from_origin_.numerical_value_ *= 2; // !!!!!
```



Structural types

```
19 - auto hours = units::quantity_cast<units::isq::si::hour>(minutes);
20 - if(hours>0*h) {
21 -     if (hours < 10*h)
13 + auto hours = minutes.force_in(u_h);
14 + if(hours>0*u_h) {
15 +     if (hours < 10*u_h)
22 16         result << "0";
23 -     result << std::to_string(hours.number());
17 +     result << std::to_string(hours.numerical_value_);
```



Mateusz Pusz @mateusz.pusz · Oct 16, 2023

Please don't use member data directly. This is the implementation detail and should not be used directly.



Mateusz Pusz @mateusz.pusz · Oct 16, 2023

It seems I have to obfuscate the name more to discourage people from using it.



[Redacted] Nov 09, 2023

Don't know if obfuscation would be effective. Maybe a short doxygen comment the discourages usage? Such a comment would pop up in must IDEs...

Owner

Structural types

```
template<Reference auto R,  
        RepresentationOf<get_quantity_spec(R)> Rep = double>  
class quantity {  
public:  
    Rep numerical_value_is_an_implementation_detail_; ///  
    // ...  
};
```

```
template<Reference auto R,  
        PointOriginFor<get_quantity_spec(R)> auto PO = default_point_origin(R),  
        RepresentationOf<get_quantity_spec(R)> Rep = double>  
class quantity_point {  
public:  
    quantity_type quantity_from_origin_is_an_implementation_detail_; ///  
    // ...  
};
```

Impact of NTTP restrictions

- Authors of C++ classes that want them to be usable as NTTPs **have to sacrifice encapsulation**

Impact of NTTP restrictions

- Authors of C++ classes that want them to be usable as NTTPs **have to sacrifice encapsulation**
- Many types from the C++ Standard Library **can't be passed as NTTPs today**
 - e.g., `std::optional`, `std::variant`, `std::tuple`, `std::span`, `std::vector`, ...

Impact of NTTP restrictions

- Authors of C++ classes that want them to be usable as NTTPs **have to sacrifice encapsulation**
- Many types from the C++ Standard Library **can't be passed as NTTPs today**
 - e.g., `std::optional`, `std::variant`, `std::tuple`, `std::span`, `std::vector`, ...
- NTTP-compatible wrapper types (e.g., `quantity<R, Rep>`) **work as long as Rep is also a structural type**
 - e.g., `quantity<si::metre, uncertainty<double>>` will not work if `uncertainty<double>` members are private

Impact of NTTP restrictions

- Authors of C++ classes that want them to be usable as NTTPs **have to sacrifice encapsulation**
- Many types from the C++ Standard Library **can't be passed as NTTPs today**
 - e.g., `std::optional`, `std::variant`, `std::tuple`, `std::span`, `std::vector`, ...
- NTTP-compatible wrapper types (e.g., `quantity<R, Rep>`) **work as long as Rep is also a structural type**
 - e.g., `quantity<si::metre, uncertainty<double>>` will not work if `uncertainty<double>` members are private
- **P3094: `std::basic_fixed_string` proposal not in C++26** because of hopes to get `std::inplace_string`
 - as of today `std::inplace_string` can't work properly as a structural type

P3380 Extending support for class types as non-type template parameters

```
template<Reference auto R,  
        RepresentationOf<get_quantity_spec(R)> Rep = double>  
class quantity {  
    Rep numerical_value_;  
    auto to_meta_representation(std::meta::serializer&) const = default;  
    static auto from_meta_representation(std::meta::deserializer&) = default;  
public:  
    // ...  
};
```

```
template<Reference auto R,  
        PointOriginFor<get_quantity_spec(R)> auto P0 = default_point_origin(R),  
        RepresentationOf<get_quantity_spec(R)> Rep = double>  
class quantity_point {  
    quantity_type quantity_from_origin_;  
    auto to_meta_representation(std::meta::serializer&) const = default;  
    static auto from_meta_representation(std::meta::deserializer&) = default;  
public:  
    // ...  
};
```

What is wrong?

CONCEPTS

```
template<Quantity Q>  
void foo(Q&& q);
```

What is wrong?

CONCEPTS

```
template<Quantity Q>  
void foo(Q&& q);
```

Q from the forwarding reference may include **&**, **const**, and **volatile**.

Lack of template parameters composition

CONCEPTS

```
template<typename Q>  
    requires Quantity<std::remove_cvref_t<Q>>  
void foo(Q&& q);
```

Lack of template parameters composition

CONCEPTS

```
template<typename Q>  
    requires Quantity<std::remove_cvref_t<Q>>  
void foo(Q&& q);
```

```
template<std::ranges::range R>  
    requires awaitable<std::ranges::range_value_t<R>>  
awaitable auto when_all(const R& awaitables);
```

Lack of template parameters composition

CONCEPTS

```
template<typename Q>  
    requires Quantity<std::remove_cvref_t<Q>>  
void foo(Q&& q);
```

```
template<std::ranges::range R>  
    requires awaitable<std::ranges::range_value_t<R>>  
awaitable auto when_all(const R& awaitables);
```

Often constraints terse or shorthand notations are not enough and additional **requires** clauses are needed to constrain template parameters.

Lack of template parameters composition

VARIABLE TEMPLATES

```
template<typename T>
inline constexpr bool treat_as_floating_point = std::is_floating_point_v<T>;

template<>
inline constexpr bool treat_as_floating_point<my_float> = true;
```

Lack of template parameters composition

VARIABLE TEMPLATES

```
template<typename T>
inline constexpr bool treat_as_floating_point = std::is_floating_point_v<T>;

template<>
inline constexpr bool treat_as_floating_point<my_float> = true;
```

```
template<template<typename> typename Pred, typename... Ts>
constexpr bool all_of = (... && Pred<Ts>::value);
```


Lack of template parameters composition

VARIABLE TEMPLATES

```
template<typename T>
inline constexpr bool treat_as_floating_point = std::is_floating_point_v<T>;

template<>
inline constexpr bool treat_as_floating_point<my_float> = true;
```

```
template<template<typename> typename Pred, typename... Ts>
constexpr bool all_of = (... && Pred<Ts>::value);
```

Only class template templates allowed in the C++ language.

Lack of template parameters composition

VARIABLE TEMPLATES

```
template<typename T>
inline constexpr bool treat_as_floating_point = std::is_floating_point_v<T>;

template<>
inline constexpr bool treat_as_floating_point<my_float> = true;
```

```
template<template<typename> typename Pred, typename... Ts>
constexpr bool all_of = (... && Pred<Ts>::value);
```

```
template<typename T>
struct treat_as_floating_point_t : std::bool_constant<treat_as_floating_point<T>> {};

static_assert(all_of<treat_as_floating_point_t, double, float, long double>);
static_assert(!all_of<treat_as_floating_point_t, int, double, float, long double>);
static_assert(!all_of<treat_as_floating_point_t, double, float, long, double>);
```

Lack of template parameters composition

VARIABLE TEMPLATES

```
template<typename T>
inline constexpr bool treat_as_floating_point = std::is_floating_point_v<T>;

template<>
inline constexpr bool treat_as_floating_point<my_float> = true;
```

```
template<template<typename> typename Pred, typename... Ts>
constexpr bool all_of = (... && Pred<Ts>::value);
```

```
template<typename T>
struct treat_as_floating_point_t : std::bool_constant<treat_as_floating_point<T>> {};

static_assert(all_of<treat_as_floating_point_t, double, float, long double>);
static_assert(all_of<treat_as_floating_point_t, int, double, float, long double>);
```

More to type, extraneous abstractions, slower to compile.

P2841 Concept and variable-template template-parameters (C++26)

BEFORE

```
template<typename Q>  
    requires Quantity<std::remove_cvref_t<Q>>  
void foo(Q&& q);
```

P2841 Concept and variable-template template-parameters (C++26)

BEFORE

```
template<typename Q>  
    requires Quantity<std::remove_cvref_t<Q>>  
void foo(Q&& q);
```

AFTER

```
template<typename T, template<typename> concept C>  
concept decays_to = C<std::decay_t<T>>;
```

P2841 Concept and variable-template template-parameters (C++26)

BEFORE

```
template<typename Q>  
    requires Quantity<std::remove_cvref_t<Q>>  
void foo(Q&& q);
```

AFTER

```
template<typename T, template<typename> concept C>  
concept decays_to = C<std::decay_t<T>>;
```

```
void foo(decays_to<Quantity> auto&& q);
```

P2841 Concept and variable-template template-parameters (C++26)

BEFORE

```
template<std::ranges::range R>  
    requires awaitable<std::ranges::range_value_t<R>>  
awaitable auto when_all(const R& awaitables);
```

P2841 Concept and variable-template template-parameters (C++26)

BEFORE

```
template<std::ranges::range R>  
    requires awaitable<std::ranges::range_value_t<R>>  
awaitable auto when_all(const R& awaitables);
```

AFTER

```
template<typename T, template<typename> concept C>  
concept range_of = std::ranges::range<T> && C<std::ranges::range_value_t<T>>;
```


P2841 Concept and variable-template template-parameters (C++26)

BEFORE

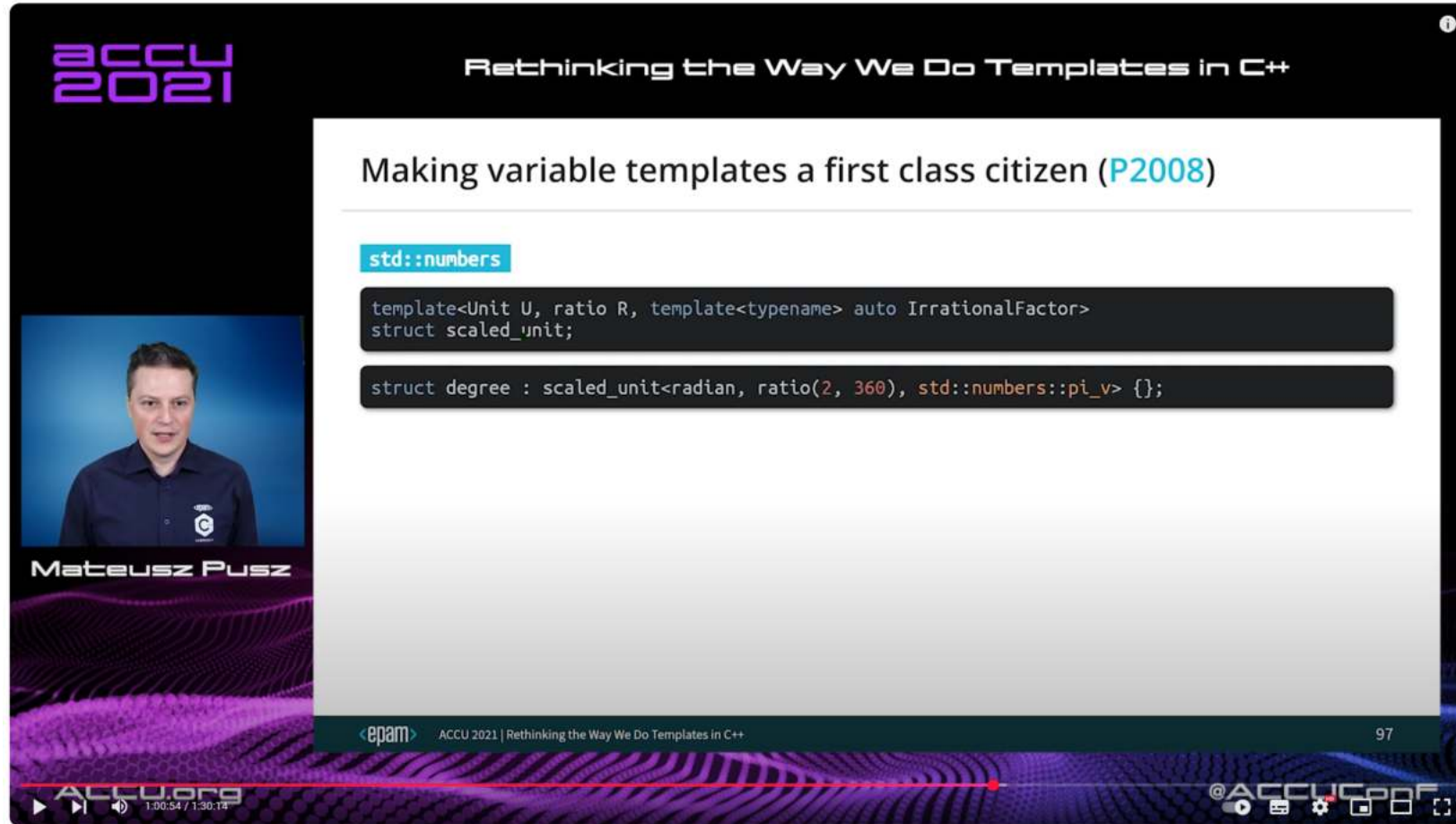
```
template<std::ranges::range R>  
    requires awaitable<std::ranges::range_value_t<R>>  
awaitable auto when_all(const R& awaitables);
```

AFTER

```
template<typename T, template<typename> concept C>  
concept range_of = std::ranges::range<T> && C<std::ranges::range_value_t<T>>;
```

```
awaitable auto when_all(const range_of<awaitable> auto& awaitables);
```

Variable template template



accu 2021

Rethinking the Way We Do Templates in C++

Making variable templates a first class citizen (P2008)

`std::numbers`

```
template<Unit U, ratio R, template<typename> auto IrrationalFactor>
struct scaled_unit;

struct degree : scaled_unit<radian, ratio(2, 360), std::numbers::pi_v> {};
```

Mateusz Pusz

97

1:00:54 / 1:30:14

@ACCUConF

Rethinking the Way We Do Templates in C++ - Mateusz Pusz [ACCU 2021]

P2841 Concept and variable-template template-parameters (C++26)

BEFORE

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of = (... && Pred<Ts>::value);
```

```
template<typename T>  
struct treat_as_floating_point_t : std::bool_constant<treat_as_floating_point<T>> {};  
  
static_assert(all_of<treat_as_floating_point_t, double, float, long double>);  
static_assert(!all_of<treat_as_floating_point_t, int, double, float, long double>);  
static_assert(!all_of<treat_as_floating_point_t, double, float, long, double>);
```

P2841 Concept and variable-template template-parameters (C++26)

BEFORE

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of = (... && Pred<Ts>::value);
```

```
template<typename T>  
struct treat_as_floating_point_t : std::bool_constant<treat_as_floating_point<T>> {};  
  
static_assert(all_of<treat_as_floating_point_t, double, float, long double>);  
static_assert(!all_of<treat_as_floating_point_t, int, double, float, long double>);  
static_assert(!all_of<treat_as_floating_point_t, double, float, long, double>);
```

AFTER

```
template<template<typename> auto Pred, typename... Ts>  
constexpr bool all_of = (... && Pred<Ts>);
```

```
static_assert(all_of<treat_as_floating_point, double, float, long double>);  
static_assert(!all_of<treat_as_floating_point, int, double, float, long double>);  
static_assert(!all_of<treat_as_floating_point, double, float, long, double>);
```

Template Parameter Kinds

BEFORE

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of = (... && Pred<Ts>::value);
```

AFTER

```
template<template<typename> auto Pred, typename... Ts>  
constexpr bool all_of = (... && Pred<Ts>);
```

Template Parameter Kinds

BEFORE

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of = (... && Pred<Ts>::value);
```

AFTER

```
template<template<typename> auto Pred, typename... Ts>  
constexpr bool all_of = (... && Pred<Ts>);
```

Why do I have to choose? Can't I have both?

Template Parameter Kinds

Templates are parameterized by *one* or *more* template parameters

Template Parameter Kinds

Templates are parameterized by *one* or *more* template parameters

- **type** template parameters

```
template<typename Ptr> class smart_ptr { /* ... */ };  
smart_ptr<int> ptr;
```


Template Parameter Kinds

Templates are parameterized by *one* or *more* template parameters

- **type** template parameters

```
template<typename Ptr> class smart_ptr { /* ... */ };  
smart_ptr<int> ptr;
```

- **non-type** template parameters

```
template<typename T, size_t N> class array { /* ... */ };  
array<int, 5> a;
```

Template Parameter Kinds

Templates are parameterized by *one* or *more* template parameters

- **type** template parameters

```
template<typename Ptr> class smart_ptr { /* ... */ };  
smart_ptr<int> ptr;
```

- **non-type** template parameters

```
template<typename T, size_t N> class array { /* ... */ };  
array<int, 5> a;
```

- **template** template parameters

```
template<typename T> class my_deleter {};  
template<typename T, template<typename> typename Policy> class handle { /* ... */ };  
handle<FILE, my_deleter> h;
```

Template Parameter Kinds

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of_t_t = (... && Pred<Ts>::value);  
  
template<template<typename> auto Pred, typename... Ts>  
constexpr bool all_of_v_t = (... && Pred<Ts>);  
  
template<template<typename> concept Pred, typename... Ts>  
constexpr bool all_of_c_t = (... && Pred<Ts>);
```

Template Parameter Kinds

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of_t_t = (... && Pred<Ts>::value);  
  
template<template<typename> auto Pred, typename... Ts>  
constexpr bool all_of_v_t = (... && Pred<Ts>);  
  
template<template<typename> concept Pred, typename... Ts>  
constexpr bool all_of_c_t = (... && Pred<Ts>);
```

```
template<template<auto> typename Pred, auto... Vs>  
constexpr bool all_of_t_v = (... && Pred<Vs>::value);  
  
template<template<auto> auto Pred, auto... Vs>  
constexpr bool all_of_v_v = (... && Pred<Vs>);  
  
template<template<auto> concept Pred, auto... Vs>  
constexpr bool all_of_c_v = (... && Pred<Vs>);
```

Template Parameter Kinds

```
template<template<typename> typename Pred, typename... Ts>
constexpr bool all_of_t_t = (... && Pred<Ts>::value);

template<template<typename> auto Pred, typename... Ts>
constexpr bool all_of_v_t = (... && Pred<Ts>);

template<template<typename> concept Pred, typename... Ts>
constexpr bool all_of_c_t = (... && Pred<Ts>);
```

```
template<template<auto> typename Pred, auto... Vs>
constexpr bool all_of_t_v = (... && Pred<Vs>::value);

template<template<auto> auto Pred, auto... Vs>
constexpr bool all_of_v_v = (... && Pred<Vs>);

template<template<auto> concept Pred, auto... Vs>
constexpr bool all_of_c_v = (... && Pred<Vs>);
```

```
template<template<typename> typename Pred, auto... Vs>
constexpr bool all_of_t_tv = (... && Pred<decltype(Vs)>::value);

template<template<typename> auto Pred, auto... Vs>
constexpr bool all_of_v_tv = (... && Pred<decltype(Vs)>);

template<template<typename> concept Pred, auto... Vs>
constexpr bool all_of_c_tv = (... && Pred<decltype(Vs)>);
```

Template Parameter Kinds

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of_t_t = (... && Pred<Ts>::value);  
  
template<template<typename> auto Pred, typename... Ts>  
constexpr bool all_of_v_t = (... && Pred<Ts>);  
  
template<template<typename> concept Pred, typename... Ts>  
constexpr bool all_of_c_t = (... && Pred<Ts>);
```

```
template<template<auto> typename Pred, auto... Vs>  
constexpr bool all_of_t_v = (... && Pred<Vs>::value);  
  
template<template<auto> auto Pred, auto... Vs>  
constexpr bool all_of_v_v = (... && Pred<Vs>);  
  
template<template<auto> concept Pred, auto... Vs>  
constexpr bool all_of_c_v = (... && Pred<Vs>);
```

```
template<template<typename> typename Pred, auto... Vs>  
constexpr bool all_of_t_tv = (... && Pred<decltype(Vs)>::value);  
  
template<template<typename> auto Pred, auto... Vs>  
constexpr bool all_of_v_tv = (... && Pred<decltype(Vs)>);  
  
template<template<typename> concept Pred, auto... Vs>  
constexpr bool all_of_c_tv = (... && Pred<decltype(Vs)>);
```

- Some user...

```
static_assert(all_of_oops<std::integral, int, 3, long, 7U>);
```

Template Parameter Kinds

```
template<template<typename> typename Pred, typename... Ts>  
constexpr bool all_of_t_t = (... && Pred<Ts>::value);  
  
template<template<typename> auto Pred, typename... Ts>  
constexpr bool all_of_v_t = (... && Pred<Ts>);  
  
template<template<typename> concept Pred, typename... Ts>  
constexpr bool all_of_c_t = (... && Pred<Ts>);
```

```
template<template<auto> typename Pred, auto... Vs>  
constexpr bool all_of_t_v = (... && Pred<Vs>::value);  
  
template<template<auto> auto Pred, auto... Vs>  
constexpr bool all_of_v_v = (... && Pred<Vs>);  
  
template<template<auto> concept Pred, auto... Vs>  
constexpr bool all_of_c_v = (... && Pred<Vs>);
```

```
template<template<typename> typename Pred, auto... Vs>  
constexpr bool all_of_t_tv = (... && Pred<decltype(Vs)>::value);  
  
template<template<typename> auto Pred, auto... Vs>  
constexpr bool all_of_v_tv = (... && Pred<decltype(Vs)>);  
  
template<template<typename> concept Pred, auto... Vs>  
constexpr bool all_of_c_tv = (... && Pred<decltype(Vs)>);
```

- Some user...

```
static_assert(all_of_oops<std::integral, int, 3, long, 7U>);
```



Universal Template Parameter Kind

Rethinking the Way We Do Templates in C++ even more

Imagine a Universal Template Parameter Kind (P1985)

A template parameter placeholder that can be replaced with any kind of a template parameter (type, non-type, template)

EXAMPLE

```
template<template auto>  
class X;
```

- takes **any kind** of template parameter

```
template<template auto...>  
class Y;
```

- takes **any number and any kind** of template parameters

76

Rethinking the Way We Do Templates in C++ even more - Mateusz Pusz [C++ on Sea 2020]

P2989 A Simple Approach to Universal Template Parameters

```
template<template<universal template> universal template Pred, universal template... Args>  
constexpr bool all_of = (... && check(Pred, Args));
```

P2989 A Simple Approach to Universal Template Parameters

```
template<template<universal template> universal template Pred, universal template Arg>  
constexpr bool check = check_impl<Pred, Arg>;
```

```
template<template<universal template> universal template Pred, universal template... Args>  
constexpr bool all_of = (... && check(Pred, Args));
```

P2989 A Simple Approach to Universal Template Parameters

```
template<template<universal template> universal template Pred, universal template Arg>
constexpr bool check = check_impl<Pred, Arg>;

template<template<typename> universal template Pred, auto Arg>
constexpr bool check<Pred, Arg> = check_impl<Pred, decltype(Arg)>;

template<template<universal template> universal template Pred, universal template... Args>
constexpr bool all_of = (... && check(Pred, Args));
```

P2989 A Simple Approach to Universal Template Parameters

```
template<template<universal template> universal template Pred, universal template Arg>  
constexpr bool check_impl = Pred<Arg>;
```

```
template<template<universal template> universal template Pred, universal template Arg>  
constexpr bool check = check_impl<Pred, Arg>;
```

```
template<template<typename> universal template Pred, auto Arg>  
constexpr bool check<Pred, Arg> = check_impl<Pred, decltype(Arg)>;
```

```
template<template<universal template> universal template Pred, universal template... Args>  
constexpr bool all_of = (... && check(Pred, Args));
```

P2989 A Simple Approach to Universal Template Parameters

```
template<template<universal template> universal template Pred, universal template Arg>
constexpr bool check_impl = Pred<Arg>;

template<template<universal template> typename Pred, universal template Arg>
constexpr bool check_impl<Pred, Arg> = Pred<Arg>::value;

template<template<universal template> universal template Pred, universal template Arg>
constexpr bool check = check_impl<Pred, Arg>;

template<template<typename> universal template Pred, auto Arg>
constexpr bool check<Pred, Arg> = check_impl<Pred, decltype(Arg)>;

template<template<universal template> universal template Pred, universal template... Args>
constexpr bool all_of = (... && check(Pred, Args));
```

P2989 A Simple Approach to Universal Template Parameters

```
template<template<universal template> universal template Pred, universal template Arg>
constexpr bool check_impl = Pred<Arg>;

template<template<universal template> typename Pred, universal template Arg>
constexpr bool check_impl<Pred, Arg> = Pred<Arg>::value;

template<template<universal template> universal template Pred, universal template Arg>
constexpr bool check = check_impl<Pred, Arg>;

template<template<typename> universal template Pred, auto Arg>
constexpr bool check<Pred, Arg> = check_impl<Pred, decltype(Arg)>;

template<template<universal template> universal template Pred, universal template... Args>
constexpr bool all_of = (... && check(Pred, Args));
```

```
static_assert(all_of<std::is_scalar, double, float, long double>);
static_assert(all_of<treat_as_floating_point, double, float, long double>);
static_assert(all_of<non_negative, 1, 3.14, 2U>);
static_assert(all_of<std::regular, double, std::pair<int, long>, std::string_view>);
static_assert(all_of<std::integral, int, 3, long, 7U>);
static_assert(all_of<Unit, second, metre, metre / second>);
static_assert(all_of<Quantity, 42 * m, isq::height(42 * m)>);
```

specialization_of

```
template<typename T, template<universal template...> typename Templ>  
constexpr bool is_specialization_of = false;
```

specialization_of

```
template<typename T, template<universal template...> typename Templ>  
constexpr bool is_specialization_of = false;  
  
template<template<universal template...> typename Templ, universal template... Params>  
constexpr bool is_specialization_of<Templ<Params...>, Templ> = true;
```


specialization_of

```
template<typename T, template<universal template...> typename Templ>  
constexpr bool is_specialization_of = false;  
  
template<template<universal template...> typename Templ, universal template... Params>  
constexpr bool is_specialization_of<Templ<Params...>, Templ> = true;  
  
template<typename T, template<universal template...> typename Templ>  
concept bool specialization_of = is_specialization_of<T, Templ>;
```

specialization_of

```
template<typename T, template<universal template...> typename Templ>
constexpr bool is_specialization_of = false;

template<template<universal template...> typename Templ, universal template... Params>
constexpr bool is_specialization_of<Templ<Params...>, Templ> = true;

template<typename T, template<universal template...> typename Templ>
concept bool specialization_of = is_specialization_of<T, Templ>;

static_assert(specialization_of<some_range, std::vector>);
static_assert(specialization_of<my_array, std::array>);
```

derived_from_specialization_of

```
template<typename T, template<universal template...> typename Templ>  
concept bool derived_from_specialization_of = is_derived_from_specialization_of<T, Templ>;
```

derived_from_specialization_of

```
template<typename T, template<universal template...> typename Templ>
constexpr bool is_derived_from_specialization_of =
    requires(T* t) { to_specialization_of<Templ>(t); };

template<typename T, template<universal template...> typename Templ>
concept bool derived_from_specialization_of = is_derived_from_specialization_of<T, Templ>;
```

derived_from_specialization_of

```
template<template<universal template...> typename Templ, universal template... Params>
void to_specialization_of(const volatile Templ<Params...>*);

template<typename T, template<universal template...> typename Templ>
constexpr bool is_derived_from_specialization_of =
    requires(T* t) { to_specialization_of<Templ>(t); };

template<typename T, template<universal template...> typename Templ>
concept bool derived_from_specialization_of = is_derived_from_specialization_of<T, Templ>;
```

derived_from_specialization_of

```
template<template<universal template...> typename Templ, universal template... Params>
void to_specialization_of(const volatile Templ<Params...>*);

template<typename T, template<universal template...> typename Templ>
constexpr bool is_derived_from_specialization_of =
    requires(T* t) { to_specialization_of<Templ>(t); };

template<typename T, template<universal template...> typename Templ>
concept bool derived_from_specialization_of = is_derived_from_specialization_of<T, Templ>;

static_assert(derived_from_specialization_of<type, quantity>);
```

Derived unit without UTP

Rethinking the Way We Do Templates in C++ even more

Mateusz Pusz

What about class templates?

CONCEPTS

```
template<typename Child, Dimension Dim, basic_fixed_string Symbol, PrefixType PT>
struct named_coherent_derived_unit;

template<typename Child, Dimension Dim>
struct coherent_derived_unit;

template<typename Child, Dimension Dim, basic_fixed_string Symbol, Ratio R, PrefixType PT = no_prefix>
struct named_scaled_derived_unit;

template<typename Child, Dimension Dim, basic_fixed_string Symbol, PrefixType PT, Unit U, Unit... Us>
struct named_deduced_derived_unit;

template<typename Child, Dimension Dim, Unit U, Unit... Us>
    requires U::is_named && (Us::is_named && ... && true)
struct deduced_derived_unit;

template<typename Child, Prefix P, Unit U>
    requires (!std::same_as<typename U::prefix_type, no_prefix>)
struct prefixed_derived_unit;
```

81

C++ on Sea 2020 | Rethinking the Way We Do Templates in C++

Rethinking the Way We Do Templates in C++ even more - Mateusz Pusz [C++ on Sea 2020]

Derived unit with UTP

Rethinking the Way We Do Templates in C++ even more



Mateusz Pusz



What about class templates?

CLASS TEMPLATE PARTIAL SPECIALIZATION OVERLOADING

```
template<template auto...>
struct derived_unit;

template<typename Child, Dimension Dim, basic_fixed_string Symbol, PrefixType PT>
struct derived_unit<Child, Dim, Symbol, PT>;

template<typename Child, Dimension Dim>
struct derived_unit<Child, Dim>;

template<typename Child, Dimension Dim, basic_fixed_string Symbol, Ratio R, PrefixType PT = no_prefix>
struct derived_unit<Child, Dim, Symbol, R, PT>;

template<typename Child, Dimension Dim, basic_fixed_string Symbol, PrefixType PT, Unit U, Unit... Us>
struct derived_unit<Child, Dim, Symbol, PT, U, Us...>;

template<typename Child, Dimension Dim, Unit U, Unit... Us>
requires U::is_named && (Us::is_named && ... && true)
struct derived_unit<Child, Dim, U, Us...>;

template<typename Child, Prefix P, Unit U>
requires (!std::same_as<typename U::prefix_type, no_prefix>)
struct derived_unit<Child, P, U>;
```

<epam> C++ on Sea 2020 | Rethinking the Way We Do Templates in C++ 82

1:14:13 / 1:37:09

Rethinking the Way We Do Templates in C++ even more - Mateusz Pusz [C++ on Sea 2020]

P2989 A Simple Approach to Universal Template Parameters

Universal Template Parameters are the cornerstone to building truly generic interfaces.

Variable templates

Customization points based on variable templates are often faster to compile and less to type than class templates, so their usage is highly recommended.

Variable templates

Customization points based on variable templates are often faster to compile and less to type than class templates, so their usage is highly recommended.

```
template<typename T>
constexpr bool treat_as_floating_point = std::is_floating_point_v<T>;

template<typename T>
constexpr bool treat_as_floating_point<Safe<T>> = treat_as_floating_point<T>;
```

Variable templates traits

```
template<std::intmax_t Value>  
    requires(0 <= Value) && (Value < 10)  
constexpr fixed_u8string superscript_number = ???;
```

```
template<> constexpr fixed_u8string superscript_number<0> = u8"0";  
template<> constexpr fixed_u8string superscript_number<1> = u8"1";  
template<> constexpr fixed_u8string superscript_number<2> = u8"2";  
template<> constexpr fixed_u8string superscript_number<3> = u8"3";  
template<> constexpr fixed_u8string superscript_number<4> = u8"4";  
template<> constexpr fixed_u8string superscript_number<5> = u8"5";  
template<> constexpr fixed_u8string superscript_number<6> = u8"6";  
template<> constexpr fixed_u8string superscript_number<7> = u8"7";  
template<> constexpr fixed_u8string superscript_number<8> = u8"8";  
template<> constexpr fixed_u8string superscript_number<9> = u8"9";
```

Variable templates traits

```
template<std::intmax_t Value>
    requires(0 <= Value) && (Value < 10)
constexpr fixed_u8string superscript_number = ???;

template<> constexpr fixed_u8string superscript_number<0> = u8"0";
template<> constexpr fixed_u8string superscript_number<1> = u8"1";
template<> constexpr fixed_u8string superscript_number<2> = u8"2";
template<> constexpr fixed_u8string superscript_number<3> = u8"3";
template<> constexpr fixed_u8string superscript_number<4> = u8"4";
template<> constexpr fixed_u8string superscript_number<5> = u8"5";
template<> constexpr fixed_u8string superscript_number<6> = u8"6";
template<> constexpr fixed_u8string superscript_number<7> = u8"7";
template<> constexpr fixed_u8string superscript_number<8> = u8"8";
template<> constexpr fixed_u8string superscript_number<9> = u8"9";
```

However, they are not the best fit if there is no good default value.

P2041 template = delete (Abandoned)

```
template<std::intmax_t Value>
    requires(0 <= Value) && (Value < 10)
constexpr fixed_u8string superscript_number = delete;

template<> constexpr fixed_u8string superscript_number<0> = u8"0";
template<> constexpr fixed_u8string superscript_number<1> = u8"1";
template<> constexpr fixed_u8string superscript_number<2> = u8"2";
template<> constexpr fixed_u8string superscript_number<3> = u8"3";
template<> constexpr fixed_u8string superscript_number<4> = u8"4";
template<> constexpr fixed_u8string superscript_number<5> = u8"5";
template<> constexpr fixed_u8string superscript_number<6> = u8"6";
template<> constexpr fixed_u8string superscript_number<7> = u8"7";
template<> constexpr fixed_u8string superscript_number<8> = u8"8";
template<> constexpr fixed_u8string superscript_number<9> = u8"9";
```

Spherical distance

```
template<typename T = double>
using latitude = mp_units::quantity_point<mp_units::si::degree, equator, bounded<T, -90, 90>>;

template<typename T = double>
using longitude = mp_units::quantity_point<mp_units::si::degree, prime_meridian, bounded<T, -180, 180>>;

template<typename T>
struct position {
    latitude<T> lat;
    longitude<T> lon;
};
```

Spherical distance

```
template<typename T = double>
using latitude = mp_units::quantity_point<mp_units::si::degree, equator, bounded<T, -90, 90>>;

template<typename T = double>
using longitude = mp_units::quantity_point<mp_units::si::degree, prime_meridian, bounded<T, -180, 180>>;

template<typename T>
struct position {
    latitude<T> lat;
    longitude<T> lon;
};
```

```
template<typename T>
distance spherical_distance(position<T> from, position<T> to)
{
    using namespace mp_units;
    constexpr quantity earth_radius = 6'371 * isq::radius[si::kilo<si::metre>];
    const quantity from_lat = from.lat.quantity_from_zero();
    const quantity from_lon = from.lon.quantity_from_zero();
    const quantity to_lat = to.lat.quantity_from_zero();
    const quantity to_lon = to.lon.quantity_from_zero();

    using si::sin, si::cos, si::asin, si::acos;
    const quantity central_angle = acos(sin(from_lat) * sin(to_lat) + cos(from_lat) * cos(to_lat) * cos(to_lon - from_lon));
    return quantity_cast<isq::distance>((earth_radius * central_angle).in(earth_radius.unit));
}
```


Spherical distance

```
template<typename T = double>
using latitude = mp_units::quantity_point<mp_units::si::degree, equator, bounded<T, -90, 90>>;

template<typename T = double>
using longitude = mp_units::quantity_point<mp_units::si::degree, prime_meridian, bounded<T, -180, 180>>;

template<typename T>
struct position {
    latitude<T> lat;
    longitude<T> lon;
};
```

```
template<typename T>
distance spherical_distance(position<T> from, position<T> to)
{
    using namespace mp_units;
    constexpr quantity earth_radius = 6'371 * isq::radius[si::kilo<si::metre>];
    const quantity from_lat = from.lat.quantity_from_zero();
    const quantity from_lon = from.lon.quantity_from_zero();
    const quantity to_lat = to.lat.quantity_from_zero();
    const quantity to_lon = to.lon.quantity_from_zero();

    using si::sin, si::cos, si::asin, si::acos;
    const quantity central_angle = acos(sin(from_lat) * sin(to_lat) + cos(from_lat) * cos(to_lat) * cos(to_lon - from_lon));
    return quantity_cast<isq::distance>((earth_radius * central_angle).in(earth_radius.unit));
}
```

[basic.lookup.argdep]

- For each argument type **T** in the function call, there is a **set of zero or more associated namespaces** and a set of zero or more associated entities (other than namespaces) **to be considered**

[basic.lookup.argdep]

- For each argument type **T** in the function call, there is a **set of zero or more associated namespaces** and a set of zero or more associated entities (other than namespaces) **to be considered**
- The sets of namespaces and entities are **determined in the following way**
 - if **T** is a **class template specialization**, its associated namespaces and entities also include
 - the *namespaces* and entities *associated with the types of the template arguments* provided for **template type parameters** (excluding template template parameters)
 - the templates used as template template arguments
 - the namespaces of which any template template arguments are members
 - the classes of which any member templates used as template template arguments are members

[basic.lookup.argdep]

- For each argument type **T** in the function call, there is a **set of zero or more associated namespaces** and a set of zero or more associated entities (other than namespaces) **to be considered**
- The sets of namespaces and entities are **determined in the following way**
 - if **T** is a **class template specialization**, its associated namespaces and entities also include
 - the *namespaces* and entities *associated with the types of the template arguments* provided for **template type parameters** (excluding template template parameters)
 - the templates used as template template arguments
 - the namespaces of which any template template arguments are members
 - the classes of which any member templates used as template template arguments are members

[Note 1 : Non-type template arguments do not contribute to the set of associated namespaces. —end note]

ADL vs NTPs



Matusz Pusz

Unleashing the Power of C++ Templates with mp-units
Lessons Learned and a New Library Design

think-cell

ADL rules are incomplete

ADL rules are incomplete! It was quite fine before C++20, but now we have new exciting use cases for NTPs.

<pre>quantity<si::metre / si::second> q; foo(q);</pre>	<pre>quantity<decltype(si::metre / si::second)> q; foo(q);</pre>
• Nice syntax and usability • ADL doesn't use namespaces of units for lookup	• Really inconvenient • ADL works as expected

Why so often we can't have nice things?

<epam> C++ on Sea 2023 | Unleashing the Power of C++ Templates with mp-units: Lessons learned and a new library design 97

cpsonsea.uk 1:08:01 / 1:22:54 2023

The Power of C++ Templates With mp-units: Lessons Learned & a New Library Design - Matusz Pusz 2023

ADL workarounds for `std::constant_wrapper` (P2781)

ADL workarounds for `std::constant_wrapper` (P2781)

§ 13 Design

§ 13.1 Add `constant_wrapper`

```
namespace std {  
    template<class T>  
        struct cw-fixed-value;                                // exposition only  
  
    template<cw-fixed-value X, class = typename decltype(cw-fixed-value(X))::type>  
        struct constant_wrapper;
```

ADL workarounds for `std::constant_wrapper` (P2781)

§ 13 Design

§ 13.1 Add `constant_wrapper`

```
namespace std {  
    template<class T>  
        struct cw-fixed-value;                                // exposition only  
  
    template<cw-fixed-value X, class = typename decltype(cw-fixed-value(X))::type>  
        struct constant_wrapper;
```

```
auto f = std::cw<"foo">;  
std::cout << f << "\n";
```

The stream insertion breaks without the unnamed parameter, which is `char const [4]`, and which in turn pulls the proper `operator<<` into consideration during ADL. Note that this ADL support is imperfect. An earlier version of the paper showed using `std::cw<strlit("foo")>` in a stream insertion operation, where `strlit` is a structural type that contains the bytes that comprise "foo":

P2822 Providing user control of associated entities of class types

FIXING ADL FOR NTTPS

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>  
class quantity namespace(decltype(X));
```

P2822 Providing user control of associated entities of class types

FIXING ADL FOR NTTPS

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>  
class quantity namespace(decltype(X));
```

IMPROVING COMPILE-TIMES

```
template<typename Source, typename Func>  
class then_sender namespace();
```

P2822 Providing user control of associated entities of class types

FIXING ADL FOR NTTPS

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>  
class quantity namespace(decltype(X));
```

IMPROVING COMPILE-TIMES

```
template<typename Source,typename Func>  
class then_sender namespace();
```

NARROWING DOWN THE SET OF ASSOCIATED ENTITIES

```
template<typename T,typename Allocator = std::allocator<T>>  
class array_map namespace(T);
```

Sorting types at compile time

```
constexpr unit auto u1 = si::kilo<si::gram> * si::metre;  
constexpr unit auto u2 = si::metre * si::kilo<si::gram>;
```

Sorting types at compile time

```
constexpr unit auto u1 = si::kilo<si::gram> * si::metre;  
constexpr unit auto u2 = si::metre * si::kilo<si::gram>;
```

```
static_assert(std::is_same_v<decltype(u1), decltype(u2)>);
```

Sorting types at compile time

```
constexpr unit auto u1 = si::kilo<si::gram> * si::metre;  
constexpr unit auto u2 = si::metre * si::kilo<si::gram>;
```

```
static_assert(std::is_same_v<decltype(u1), decltype(u2)>);
```

```
derived_unit<si::kilo_<si::gram>, si::metre>
```

Sorting types at compile time

```
template<typename Lhs, typename Rhs, template<typename, typename> typename Pred>  
using expr_less = expr_less_impl<Lhs, Rhs, Pred>;
```

Sorting types at compile time

```
template<typename Lhs, typename Rhs, template<typename, typename> typename Pred>  
struct expr_less_impl : Pred<expr_type<Lhs>, expr_type<Rhs>> {};  
  
template<typename Lhs, typename Rhs, template<typename, typename> typename Pred>  
using expr_less = expr_less_impl<Lhs, Rhs, Pred>;
```


Sorting types at compile time

```
template<typename Lhs, typename Rhs, template<typename, typename> typename Pred>  
struct expr_less_impl : Pred<expr_type<Lhs>, expr_type<Rhs>> {};
```

```
template<typename Lhs, typename Rhs, template<typename, typename> typename Pred>  
using expr_less = expr_less_impl<Lhs, Rhs, Pred>;
```

```
template<typename T>  
[[nodiscard]] constexpr std::string_view type_name();
```

```
template<typename Lhs, typename Rhs>  
struct type_name_less : std::bool_constant<type_name<Lhs>() < type_name<Rhs>()> {};
```

```
template<typename T1, typename T2>  
using type_list_name_less = expr_less<T1, T2, type_name_less>;
```

Issues with sorting types at compile time

```
template<typename T>
[[nodiscard]] constexpr std::string_view type_name()
{
    std::string_view name, prefix, suffix;
#ifdef __clang__
    name = __PRETTY_FUNCTION__;
    prefix = "auto type_name() [T = ";
    suffix = "]";
#elif defined(__GNUC__)
    name = __PRETTY_FUNCTION__;
    prefix = "constexpr auto type_name() [with T = ";
    suffix = "]";
#elif defined(_MSC_VER)
    name = __FUNCSIG__;
    prefix = "auto __cdecl type_name<";
    suffix = ">(void)";
#endif
    name.remove_prefix(prefix.size());
    name.remove_suffix(suffix.size());
    return name;
}
```

Issues with sorting types at compile time

```
template<typename T>
[[nodiscard]] constexpr std::string_view type_name()
{
    std::string_view name, prefix, suffix;
#ifdef __clang__
    name = __PRETTY_FUNCTION__;
    prefix = "auto type_name() [T = ";
    suffix = "]";
#elif defined(__GNUC__)
    name = __PRETTY_FUNCTION__;
    prefix = "constexpr auto type_name() [with T = ";
    suffix = "]";
#elif defined(_MSC_VER)
    name = __FUNCSIG__;
    prefix = "auto __cdecl type_name<";
    suffix = ">(void)";
#endif
}
```

Non-C++ standard hack 🤪

Issues with sorting types at compile time

```
constexpr unit auto u1 = si::kilo<si::gram> * si::metre;  
constexpr unit auto u2 = si::metre * si::kilo<si::gram>;
```

```
static_assert(std::is_same_v<decltype(u1), decltype(u2)>);
```

```
derived_unit<si::kilo_<si::gram>, si::metre>
```

Issues with sorting types at compile time

```
constexpr unit auto u1 = si::kilo<si::gram> * si::metre;  
constexpr unit auto u2 = si::metre * si::kilo<si::gram>;
```

```
static_assert(std::is_same_v<decltype(u1), decltype(u2)>);
```

```
derived_unit<si::kilo_<si::gram>, si::metre>
```

```
namespace {  
  
inline constexpr struct cows final : named_unit<"cows", one> {} cows;  
  
}
```

```
constexpr unit auto u3 = cows * si::metre;
```

Issues with sorting types at compile time

```
constexpr unit auto u1 = si::kilo<si::gram> * si::metre;  
constexpr unit auto u2 = si::metre * si::kilo<si::gram>;
```

```
static_assert(std::is_same_v<decltype(u1), decltype(u2)>);
```

derived_unit<si::kilo_<si::gram>, si::metre>

```
namespace {  
  
inline constexpr struct cows final : named_unit<"cows", one> {} cows;  
  
}
```

```
constexpr unit auto u3 = cows * si::metre;
```

GCC

derived_unit<si::metre, {anonymous}::cows>

Issues with sorting types at compile time

```
constexpr unit auto u1 = si::kilo<si::gram> * si::metre;  
constexpr unit auto u2 = si::metre * si::kilo<si::gram>;
```

```
static_assert(std::is_same_v<decltype(u1), decltype(u2)>);
```

```
derived_unit<si::kilo_<si::gram>, si::metre>
```

```
namespace {  
  
inline constexpr struct cows final : named_unit<"cows", one> {} cows;  
  
}
```

```
constexpr unit auto u3 = cows * si::metre;
```

GCC

```
derived_unit<si::metre, {anonymous}::cows>
```

CLANG

```
derived_unit<(anonymous namespace)::cows, si::metre>
```

P2830 Standardized Constexpr Type Ordering (C++26?)

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
struct expr_less_impl : Pred<expr_type<Lhs>, expr_type<Rhs>> {};
```

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
using expr_less = expr_less_impl<Lhs, Rhs, Pred>;
```

```
template<typename T1, typename T2>  
using type_list_name_less = expr_less<T1, T2, std::type_order>;
```


P2830 Standardized Constexpr Type Ordering (C++26?)

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
struct expr_less_impl : Pred<expr_type<Lhs>, expr_type<Rhs>> {};
```

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
using expr_less = expr_less_impl<Lhs, Rhs, Pred>;
```

```
template<typename T1, typename T2>  
using type_list_name_less = expr_less<T1, T2, std::type_order>;
```

- Canonicalized *type sets*
 - `std::is_same_v<typeset<A, B>, typeset<B, A, A>>`

P2830 Standardized Constexpr Type Ordering (C++26?)

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
struct expr_less_impl : Pred<expr_type<Lhs>, expr_type<Rhs>> {};
```

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
using expr_less = expr_less_impl<Lhs, Rhs, Pred>;
```

```
template<typename T1, typename T2>  
using type_list_name_less = expr_less<T1, T2, std::type_order>;
```

- Canonicalized *type sets*
 - `std::is_same_v<typeset<A, B>, typeset<B, A, A>>`
- Canonicalizing *policy-based libraries*
 - `using T1 = lib::dyn<ostreamable, iterable<int>, scalar_multiplicable<int>>;`

P2830 Standardized Constexpr Type Ordering (C++26?)

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
struct expr_less_impl : Pred<expr_type<Lhs>, expr_type<Rhs>> {};
```

```
template<typename Lhs, typename Rhs, template<typename, typename, auto...> typename Pred>  
using expr_less = expr_less_impl<Lhs, Rhs, Pred>;
```

```
template<typename T1, typename T2>  
using type_list_name_less = expr_less<T1, T2, std::type_order>;
```

- Canonicalized *type sets*
 - `std::is_same_v<typeset<A, B>, typeset<B, A, A>>`
- Canonicalizing *policy-based libraries*
 - `using T1 = lib::dyn<ostreamable, iterable<int>, scalar_multiplicable<int>>;`
- Canonicalized *variant*
 - `template<typename... Ts> using variant_for = canonicalized<std::variant, Ts...>;`

Always use constructor initializer lists

Always use constructor initializer lists

```
template<typename T>
class measurement {
public:
    using value_type = T;

    measurement() = default;

    constexpr explicit measurement(value_type val, const value_type& err = {}) : value_(std::move(val))
    {
        using namespace std;
        uncertainty_ = abs(err);
    }

    // ...

private:
    value_type value_{};
    value_type uncertainty_{};
};
```

Using declaration can't be provided in a constructor initializer list.

Proper math for wrapping types

```
template<auto R, typename Rep>
    requires requires(Rep v) { abs(v); } || requires(Rep v) { std::abs(v); }
[[nodiscard]] constexpr Quantity auto abs(const quantity<R, Rep>& q) noexcept
{
    using std::abs;
    return quantity{abs(q.numerical_value_ref_in(q.unit)), R};
}
```

Using declaration can't be provided in constraints.

Should we have CPOs for mathematical functions?

Many people work now on libraries providing custom "smart" numeric representation types/wrappers.

Should we have CPOs for mathematical functions?

Many people work now on libraries providing custom "smart" numeric representation types/wrappers.

CPOs for math functions could increase interop between various numeric-related types.

Customization Point Object (CPO)

```
template<typename T>
concept not_void = (!std::is_void_v<T>);
```

```
namespace math {
namespace abs_impl {

void abs() = delete; // poison pill

struct abs_t {
    [[nodiscard]] constexpr auto operator()(const auto& val) const
        requires requires { std::abs(val); } ||
        requires { { val.abs() } -> not_void; } || requires { { abs(val) } -> not_void; }
    {
        if constexpr (requires { val.abs(); }) return val.abs();
        else if constexpr (requires { abs(val); }) return abs(val);
        else return std::abs(val);
    }
};

} // namespace abs_impl

inline constexpr abs_impl::abs_t abs;
} // namespace math
```

Customization Point Object (CPO)

```
template<typename T>
concept not_void = (!std::is_void_v<T>);
```

```
namespace math {
namespace abs_impl {

void abs() = delete; // poison pill

struct abs_t {
    [[nodiscard]] constexpr auto operator()(const auto& val) const
        requires requires { std::abs(val); } ||
        requires { { val.abs() } -> not_void; } || requires { { abs(val) } -> not_void; }
    {
        if constexpr (requires { val.abs(); }) return val.abs();
        else if constexpr (requires { abs(val); }) return abs(val);
        else return std::abs(val);
    }
};

} // namespace abs_impl
```

More powerful than the previous simple checks.

Proper math for wrapping types

BEFORE

```
template<auto R, typename Rep>
    requires requires(Rep v) { abs(v); } || requires(Rep v) { std::abs(v); }
[[nodiscard]] constexpr Quantity auto abs(const quantity<R, Rep>& q) noexcept
{
    using std::abs;
    return quantity{abs(q.numerical_value_ref_in(q.unit)), R};
}
```

Proper math for wrapping types

BEFORE

```
template<auto R, typename Rep>
    requires requires(Rep v) { abs(v); } || requires(Rep v) { std::abs(v); }
[[nodiscard]] constexpr Quantity auto abs(const quantity<R, Rep>& q) noexcept
{
    using std::abs;
    return quantity{abs(q.numerical_value_ref_in(q.unit)), R};
}
```

AFTER

```
template<auto R, typename Rep>
    requires requires(Rep v) { math::abs(v); }
[[nodiscard]] constexpr Quantity auto abs(const quantity<R, Rep>& q) noexcept
{
    return quantity{math::abs(q.numerical_value_ref_in(q.unit)), R};
}
```

ENABLING COMPILE-TIME CODE DEBUGGING

Compile-time code debugging

Traditional debugging of `constexpr` code is hard. Debugging of `constexpr` code is impossible.

Compile-time code debugging

Traditional debugging of **constexpr** code is hard. Debugging of **constexpr** code is impossible.

```
template<QuantitySpec From, QuantitySpec To>
[[nodiscard]] constexpr specs_convertible_result convertible_impl(From from, To to)
{
    using enum specs_convertible_result;

    if constexpr (From{} == To{})
        return yes;
    else if constexpr (From::dimension != To::dimension)
        return no;
    else if constexpr (QuantityKindSpec<From> || QuantityKindSpec<To>)
        return convertible_kinds(from, to);
    else if constexpr (NamedQuantitySpec<From> && NamedQuantitySpec<To>)
        return convertible_named(from, to);
    else
        return are_ingredients_convertible(from, to);
}
```

Compile-time code debugging

Traditional debugging of `constexpr` code is hard. Debugging of `constexpr` code is impossible.

```
template<QuantitySpec From, QuantitySpec To>
[[nodiscard]] constexpr specs_convertible_result convertible_impl(From from, To to)
{
    using enum specs_convertible_result;

    if constexpr (From{} == To{})
        return yes;
    else if constexpr (From::dimension != To::dimension)
        return no;
    else if constexpr (QuantityKindSpec<From> || QuantityKindSpec<To>)
        return convertible_kinds(from, to);
    else if constexpr (NamedQuantitySpec<From> && NamedQuantitySpec<To>)
        return convertible_named(from, to);
    else
        return are_ingredients_convertible(from, to);
}
```



P2758 Emitting messages at compile time

```
template<QuantitySpec From, QuantitySpec To>
[[nodiscard]] constexpr specs_convertible_result convertible(From from, To to)
{
    using enum specs_convertible_result;

    if constexpr (From{} == To{}) {
        std::constexpr_print_str("convertible: same");
        return yes;
    }
    else if constexpr (From::dimension != To::dimension) {
        std::constexpr_print_str("convertible: different dimension");
        return no;
    }
    else if constexpr (QuantityKindSpec<From> || QuantityKindSpec<To>) {
        std::constexpr_print_str("convertible: kind comparison");
        return convertible_kinds(from, to);
    }
    else if constexpr (NamedQuantitySpec<From> && NamedQuantitySpec<To>) {
        std::constexpr_print_str("convertible: named comparison");
        return convertible_named(from, to);
    }
    else {
        std::constexpr_print_str("convertible: ingredients comparison");
        return are_ingredients_convertible(from, to);
    }
}
```

P2758 Emitting messages at compile time

```
template<QuantitySpec From, QuantitySpec To>
[[nodiscard]] constexpr specs_convertible_result convertible(From from, To to)
{
    using enum specs_convertible_result;

    if constexpr (From{} == To{}) {
        std::constexpr_print_str("mp-units-debug-prints", "convertible: same");
        return yes;
    }
    else if constexpr (From::dimension != To::dimension) {
        std::constexpr_print_str("mp-units-debug-prints", "convertible: different dimension");
        return no;
    }
    else if constexpr (QuantityKindSpec<From> || QuantityKindSpec<To>) {
        std::constexpr_print_str("mp-units-debug-prints", "convertible: kind comparison");
        return convertible_kinds(from, to);
    }
    else if constexpr (NamedQuantitySpec<From> && NamedQuantitySpec<To>) {
        std::constexpr_print_str("mp-units-debug-prints", "convertible: named comparison");
        return convertible_named(from, to);
    }
    else {
        std::constexpr_print_str("mp-units-debug-prints", "convertible: ingredients comparison");
        return are_ingredients_convertible(from, to);
    }
}
```

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code*
 - in case of translations some **format** arguments may be unused

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code* 🤖
 - in case of translations some **format** arguments may be unused

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code* 🤖 🤔
 - in case of translations some **format** arguments may be unused

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code* 🤖 🧐 🧐
 - in case of translations some **format** arguments may be unused

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code* 🤖 🧐 🧐 🤖
 - in case of translations some **format** arguments may be unused

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code* 🤖 🧐 🧐 🧐
 - in case of translations some **format** arguments may be unused
- Most users would probably like to *diagnose such issues* in their format strings

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code* 🤖 🧐 🧐 🧐
 - in case of translations some **format** arguments may be unused
- Most users would probably like to *diagnose such issues* in their format strings

```
std::constexpr_warning_str("format-too-many-args",  
                           "Format string consumed less arguments than were provided.");
```

P2758 Emitting messages at compile time

```
std::format("x={} and y=", x, y);
```

- *Valid but surprising code* 🤖 🤔 😄 🚫
 - in case of translations some **format** arguments may be unused
- Most users would probably like to *diagnose such issues* in their format strings

```
std::constexpr_warning_str("format-too-many-args",  
                           "Format string consumed less arguments than were provided.");
```

- Compilers may allow setting this warning tag from the *command line options or with pragmas*
 - enabled with **-Wformat-too-many-args**
 - disabled with **-Wno-format-too-many-args**

P2758 Emitting messages at compile time

```
constexpr int foo(int a)
{
    if (a == 0)
        std::constexpr_error_str("reject-zero", "can't call with a == 0");
    return a;
}

int x = foo(2); // OK, constant-initialized
int y = foo(0); // error: reject-zero, can't call with a == 0
```

BETTER COMPILE-TIME ERROR MESSAGES

Converting units

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    static constexpr Reference auto reference = R;
    static constexpr QuantitySpec auto quantity_spec = get_quantity_spec(reference);
    static constexpr Unit auto unit = get_unit(reference);
    using rep = Rep;

};
```

Converting units

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    static constexpr Reference auto reference = R;
    static constexpr QuantitySpec auto quantity_spec = get_quantity_spec(reference);
    static constexpr Unit auto unit = get_unit(reference);
    using rep = Rep;

    template<UnitCompatibleWith<unit, quantity_spec> ToU>
        requires ValuePreservingTo<rep, rep, unit, ToU{}>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

Example: Invalid type (clang-20)

```
quantity q = 42 * m;  
quantity q1 = q.in(isq::time);
```

error: no matching member function for call to 'in'

```
10 |     quantity q1 = q.in(isq::time);  
    |                      ~~~^
```

note: candidate template ignored: constraints not satisfied [with ToU = struct time]

```
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                                                                ^
```

note: because 'UnitCompatibleWith<mp_units::isq::time, unit, quantity_spec>' evaluated to false

```
242 |     template<UnitCompatibleWith<unit, quantity_spec> ToU>  
    |           ^
```

note: because 'mp_units::isq::time' does not satisfy 'Unit'

```
120 |     Unit<U> && Unit<decltype(FromU)> && QuantitySpec<decltype(QS)> &&  
    |     ^
```

note: because 'std::derived_from<mp_units::isq::time, unit_interface>' evaluated to false

```
45 |     concept Unit = SymbolicConstant<T> && std::derived_from<T, unit_interface>;  
    |                               ^
```

note: because 'is_base_of_v<mp_units::unit_interface, mp_units::isq::time>' evaluated to false

```
27 |     concept derived_from = is_base_of_v<_Bp, _Dp> && is_convertible_v<const volatile _Dp*, const volatile _Bp*>;  
    |                               ^
```

1 error generated.

Example: Unit of a different quantity kind (clang-20)

```
quantity q = 42 * m;  
quantity q2 = q.in(s);
```

error: no matching member function for call to 'in'

```
10 |     quantity q1 = q.in(s);  
    |                   ~~~~
```

note: candidate template ignored: constraints not satisfied [with ToU = struct second]

```
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                                     ^
```

note: because 'UnitCompatibleWith<mp_units::si::second, unit, quantity_spec>' evaluated to false

```
242 |     template<UnitCompatibleWith<unit, quantity_spec> ToU>  
    |             ^
```

note: because 'WeakUnitOf<mp_units::si::second, kind_of_<decltype(struct length{{{}}})>{{{}}}>' evaluated to false

```
121 |     WeakUnitOf<U, QS> && (interconvertible(FromU, U{}));  
    |     ^
```

note: because '!AssociatedUnit<mp_units::si::second>' evaluated to false

```
110 |     Unit<U> && QuantitySpec<decltype(QS)> && ((!AssociatedUnit<U>) || UnitOf<U, QS>);  
    |                                     ^
```

note: and 'UnitOf<mp_units::si::second, kind_of_<decltype(struct length{{{}}})>{{{}}}>' evaluated to false

```
110 |     Unit<U> && QuantitySpec<decltype(QS)> && ((!AssociatedUnit<U>) || UnitOf<U, QS>);  
    |                                     ^
```

note: because 'implicitly_convertible(get_quantity_spec(mp_units::si::second{}),

```
101 |         (implicitly_convertible(get_quantity_spec(U{}), QS));  
    |         ^
```

1 error generated.

Example: Unit of a different quantity kind (clang-20)

```
quantity q = 42 * m;  
quantity q2 = q.in(s);
```

error: no matching member function for call to 'in'

```
10 |     quantity q1 = q.in(s);  
   |                   ~~~~  
note: candidate template ignored: constraints not satisfied [with ToU = struct second]  
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
   |                   ^  
note: because 'UnitCompatibleWith<mp_units::si::second, unit, quantity_spec>' evaluated to false  
242 |     template<UnitCompatibleWith<unit, quantity_spec> ToU>  
   |             ^  
note: because 'WeakUnitOf<mp_units::si::second, kind_of_<decltype(struct length{{{}}})>{{{}}}>' evaluated to false  
121 |     WeakUnitOf<U, QS> && (interconvertible(FromU, U{}));  
   |     ^  
note: because '!AssociatedUnit<mp_units::si::second>' evaluated to false  
110 |     Unit<U> && QuantitySpec<decltype(QS)> && ((!AssociatedUnit<U>) || UnitOf<U, QS>);  
   |                   ^  
note: and 'UnitOf<mp_units::si::second, kind_of_<decltype(struct length{{{}}})>{{{}}}>' evaluated to false  
110 |     Unit<U> && QuantitySpec<decltype(QS)> && ((!AssociatedUnit<U>) || UnitOf<U, QS>);  
   |                   ^  
note: because 'implicitly_convertible(get_quantity_spec(mp_units::si::second{}),  
   |                   kind_of_<decltype(struct length{{{}}})>{{{}}})' evaluated to false`  
101 |                   (implicitly_convertible(get_quantity_spec(U{}), QS));  
   |                   ^  
1 error generated.
```

Example: Unit of a different quantity kind (clang-20)

```
quantity q = 42 * m;  
quantity q2 = q.in(s);
```

error: no matching member function for call to 'in'

```
10 |     quantity q1 = q.in(s);  
   |                   ~~~~  
note: candidate template ignored: constraints not satisfied [with ToU = struct second]  
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
   |                   ^  
note: because 'UnitCompatibleWith<mp_units::si::second, unit, quantity_spec>' evaluated to false  
242 |     template<UnitCompatibleWith<unit, quantity_spec> ToU>  
   |             ^  
note: because 'WeakUnitOf<mp_units::si::second, kind_of_<length>' evaluated to false  
121 |     WeakUnitOf<U, QS> && (interconvertible(FromU, U{}));  
   |     ^  
note: because '!AssociatedUnit<mp_units::si::second>' evaluated to false  
110 |     Unit<U> && QuantitySpec<decltype(QS)> && ((!AssociatedUnit<U>) || UnitOf<U, QS>);  
   |                   ^  
note: and 'UnitOf<mp_units::si::second, kind_of_<length>' evaluated to false  
110 |     Unit<U> && QuantitySpec<decltype(QS)> && ((!AssociatedUnit<U>) || UnitOf<U, QS>);  
   |                   ^  
note: because 'implicitly_convertible(get_quantity_spec(mp_units::si::second{}),  
   |                   kind_of_<length>)' evaluated to false`  
101 |                   (implicitly_convertible(get_quantity_spec(U{}), QS));  
   |                   ^  
1 error generated.
```

Example: Value truncation (clang-20)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: no matching member function for call to 'in'  
10 |     quantity q3 = q.in(km);  
    |                   ~~~~  
note: candidate template ignored: constraints not satisfied [with ToU = kilo_<decltype(metre{{{}}})>]  
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                   ^  
note: because 'ValuePreservingTo<rep, rep, unit, mp_units::si::kilo_<mp_units::si::metre>{}>' evaluated to false  
243 |     requires ValuePreservingTo<rep, rep, unit, ToU{}>  
    |             ^  
note: because 'int' does not satisfy 'IsFloatingPoint'  
70 |     (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&  
    |                               ^  
note: because 'treat_as_floating_point<int>' evaluated to false  
65 |     concept IsFloatingPoint = treat_as_floating_point<T>;  
    |                               ^  
note: and 'integral_conversion_factor(mp_units::si::metre{{{}}},  
71 |     mp_units::si::kilo_<mp_units::si::metre>{{{}}})' evaluated to false  
    |                               ^  
    |                               integral_conversion_factor(FromUnit, ToUnit) &&  
1 error generated.
```

Example: Value truncation (gcc-14)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

In function 'int main()':

error: no matching function for call to 'mp_units::quantity<mp_units::si::metre(), int>::in(const mp_units::si::kilo_<mp_units::si::metre>&)'

```
10 |     quantity q3 = q.in(km);  
    |                   ~~~~^~~~
```

note: candidate: 'template<class ToU> requires (UnitCompatibleWith<ToU, mp_units::quantity<R, Rep>::unit, mp_units::quantity<R, Rep>::quantity_spec>) && (ValuePreservingTo<Rep, Rep, mp_units::quantity<R, Rep>::unit, {}>) constexpr auto [requires mp_units::QuantityOf<<placeholder>, mp_units::quantity<R, Rep>::quantity_spec>] mp_units::quantity<R, Rep>::in(ToU) const [with auto [requires mp_units::Reference<<placeholder>, >] R = mp_units::si::metre(); Rep = int]'

```
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                                     ^~
```

note: template argument deduction/substitution failed:

note: constraints not satisfied

In substitution of 'template<class ToU> requires (UnitCompatibleWith<ToU, mp_units::quantity<R, Rep>::unit, mp_units::quantity<R, Rep>::quantity_spec>) && (ValuePreservingTo<Rep, Rep, mp_units::quantity<R, Rep>::unit, {}>) constexpr auto [requires mp_units::QuantityOf<<placeholder>, mp_units::quantity<R, Rep>::quantity_spec>] mp_units::quantity<mp_units::si::metre(), int>::in(ToU) const [with ToU = mp_units::si::metre()]':
required from here

```
10 |     quantity q3 = q.in(km);  
    |                   ~~~~^~~~
```

required for the satisfaction of 'ValuePreservingTo<Rep, Rep, mp_units::quantity<R, Rep>::unit, ToU{}>'

[with Rep = int; ToU = mp_units::si::kilo_<mp_units::si::metre>; R = mp_units::si::metre{}]

note: no operand of the disjunction is satisfied

```
70 |                                     (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&  
    |                                     ~~~~~^~~~~~  
71 |                                     integral_conversion_factor(FromUnit, ToUnit) &&  
    |                                     ~~~~~~  
72 |                                     !overflows_non_zero_values<ToRep>(FromUnit, ToUnit)));  
    |                                     ~~~~~~
```

cc1plus: note: set '-fconcepts-diagnostics-depth=' to at least 2 for more detail

Converting units

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<UnitCompatibleWith<unit, quantity_spec> ToU>
        requires ValuePreservingTo<rep, rep, unit, ToU{}>
        [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
        {
            return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
        }
        // ...
};
```

Converting units

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        static_assert(UnitCompatibleWith<ToU, unit, quantity_spec>, "incompatible unit");
        static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>, "not-value preserving");
        return quantity<make_reference(quantity_spec, ToU{})>, Rep{*this};
    }
    // ...
};
```

Example: Value truncation (clang-20)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: static assertion failed: not-value preserving  
253 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>, "not-value preserving");  
    |                   ^~~~~~  
note: in instantiation of function template specialization 'mp_units::quantity<mp_units::si::metre{{{}}}, int>::  
    | in<mp_units::si::kilo_<mp_units::si::metre>>' requested here  
10 |     quantity q3 = q.in(km);  
    |                   ^  
note: because 'ValuePreservingTo<rep, rep, unit, mp_units::si::kilo_<mp_units::si::metre>{}>' evaluated to false  
253 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>, "not-value preserving");  
    |                   ^  
note: because 'int' does not satisfy 'IsFloatingPoint'  
70 |     (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&  
    |                                     ^  
note: because 'treat_as_floating_point<int>' evaluated to false  
65 |     concept IsFloatingPoint = treat_as_floating_point<T>;  
    |                               ^  
note: and 'integral_conversion_factor(mp_units::si::metre{{{}}},  
    | mp_units::si::kilo_<mp_units::si::metre>{{{}}}, {{{}}})' evaluated to false  
71 |     integral_conversion_factor(FromUnit, ToUnit) &&  
    |                               ^  
...  
2 errors generated.
```

P2741 user-generated `static_assert` messages (C++26)

```
constexpr std::string constexpr_format(auto fmt, auto&&... args)
{
    std::string text;
    fmt::format_to(std::back_inserter(text), fmt, std::forward<decltype(args)>(args)...);
    return text;
};
```


P2741 user-generated static_assert messages (C++26)

```
constexpr std::string constexpr_format(auto fmt, auto&&... args)
{
    std::string text;
    fmt::format_to(std::back_inserter(text), fmt, std::forward<decltype(args)>(args)...);
    return text;
};
```

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        static_assert(UnitCompatibleWith<ToU, unit, quantity_spec>,
            constexpr_format(FMT_COMPILE("Unit '{}' is incompatible for '{}'",
                                         unit_symbol(ToU{}), unit_symbol(unit))),
            unit_symbol(ToU{}), unit_symbol(unit));
        static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,
            constexpr_format(
                FMT_COMPILE("Conversion from '{}' to '{}' is not value-preserving"),
                unit_symbol(unit), unit_symbol(ToU{}))),
            unit_symbol(unit), unit_symbol(ToU{}));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

Example: Value truncation (clang-20)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: static assertion failed: Conversion from 'm' to 'km' is not value-preserving  
256 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
      |                   ^~~~~~  
note: in instantiation of function template specialization 'mp_units::quantity<mp_units::si::metre{{{}}}, int>::  
      | in<mp_units::si::kilo_<mp_units::si::metre>>' requested here  
10 |     quantity q3 = q.in(km);  
    |                   ^  
note: because 'ValuePreservingTo<rep, rep, unit, mp_units::si::kilo_<mp_units::si::metre>{}>' evaluated to false  
256 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
      |                   ^  
note: because 'int' does not satisfy 'IsFloatingPoint'  
70 |     (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&  
    |                                ^  
note: because 'treat_as_floating_point<int>' evaluated to false  
65 |     concept IsFloatingPoint = treat_as_floating_point<T>;  
    |                                ^  
note: and 'integral_conversion_factor(mp_units::si::metre{{{}}},  
    | mp_units::si::kilo_<mp_units::si::metre>{{{}}}, {{{}}})' evaluated to false  
71 |     integral_conversion_factor(FromUnit, ToUnit) &&  
    |                                ^  
...  
2 errors generated.
```

P3391 constexpr std::format

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        static_assert(UnitCompatibleWith<ToU, unit, quantity_spec>,
            std::format("Unit '{}' is incompatible for '{}'", unit_symbol(ToU{}), unit_symbol(unit)));
        static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,
            std::format("Conversion from '{}' to '{}' is not value-preserving",
                unit_symbol(unit), unit_symbol(ToU{})));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

P2996 Reflection for C++26

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        static_assert(UnitCompatibleWith<ToU, unit, quantity_spec>,
            std::format("Unit '{}' is incompatible for '{}'", unit_symbol(ToU{}), unit_symbol(unit)));
        static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,
            std::format("Conversion from '{}' to '{}' is not value-preserving for '{}' representation type",
                unit_symbol(unit), unit_symbol(ToU{}), display_string_of(dealias^^rep)));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

Example: Value truncation (clang-20)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: static assertion failed: Conversion from 'm' to 'km' is not value-preserving for 'int' representation type  
256 |         static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
      |                        ^~~~~~  
note: in instantiation of function template specialization 'mp_units::quantity<mp_units::si::metre{{{}}}, int>::  
      |         in<mp_units::si::kilo_<mp_units::si::metre>>' requested here  
10  |     quantity q3 = q.in(km);  
      |                     ^  
note: because 'ValuePreservingTo<rep, rep, unit, mp_units::si::kilo_<mp_units::si::metre>{}>' evaluated to false  
256 |         static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
      |                        ^  
note: because 'int' does not satisfy 'IsFloatingPoint'  
70  |                                     (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&  
      |                                     ^  
note: because 'treat_as_floating_point<int>' evaluated to false  
65  |     concept IsFloatingPoint = treat_as_floating_point<T>;  
      |                               ^  
note: and 'integral_conversion_factor(mp_units::si::metre{{{}}},  
      |         mp_units::si::kilo_<mp_units::si::metre>{{{}}}, {{{}}})' evaluated to false  
71  |                                     integral_conversion_factor(FromUnit, ToUnit) &&  
      |                                     ^  
...  
2 errors generated.
```

Problems with static_assert()

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: static assertion failed: Conversion from 'm' to 'km' is not value-preserving for 'int' representation type  
256 |         static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
      |                        ^~~~~~  
note: in instantiation of function template specialization 'mp_units::quantity<mp_units::si::metre{{{}}}, int>::  
      |         in<mp_units::si::kilo_<mp_units::si::metre>>' requested here  
10  |         quantity q3 = q.in(km);  
      |                        ^  
note: because 'ValuePreservingTo<rep, rep, unit, mp_units::si::kilo_<mp_units::si::metre>{}>' evaluated to false  
256 |         static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
      |                        ^  
note: because 'int' does not satisfy 'IsFloatingPoint'  
70  |                                (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&  
      |                                ^  
note: because 'treat_as_floating_point<int>' evaluated to false  
65  |     concept IsFloatingPoint = treat_as_floating_point<T>;  
      |                                ^  
note: and 'integral_conversion_factor(mp_units::si::metre{{{}}},  
      |         mp_units::si::kilo_<mp_units::si::metre>{{{}}}, {{{}}})' evaluated to false  
71  |                                integral_conversion_factor(FromUnit, ToUnit) &&  
      |                                ^  
...  
2 errors generated.
```

Problems with static_assert()

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: static assertion failed: Conversion from 'm' to 'km' is not value-preserving for 'int' representation type  
256 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
    |                   ^~~~~~  
note: in instantiation of function template specialization 'mp_units::quantity<mp_units::si::metre{{{}}}, int>::  
    |     in<mp_units::si::kilo_<mp_units::si::metre>>' requested here  
10 |     quantity q3 = q.in(km);  
    |                   ^  
note: because 'ValuePreservingTo<rep, rep, unit, mp_units::si::kilo_<mp_units::si::metre>{}>' evaluated to false  
256 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,  
    |                   ^  
note: because 'int' does not satisfy 'IsFloatingPoint'  
70 |     (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&  
    |     ^  
note: because 'treat_as_floating_point<int>' evaluated to false  
65 |     concept IsFloatingPoint = treat_as_floating_point<T>;  
    |                             ^  
note: and 'integral_conversion_factor(mp_units::si::metre{{{}}},  
    |     mp_units::si::kilo_<mp_units::si::metre>{{{}}}, {{{}}})' evaluated to false  
71 |     integral_conversion_factor(FromUnit, ToUnit) &&  
    |     ^  
error: no matching constructor for initialization of 'quantity<make_reference(quantity_spec, kilo_<metre>{}), int>'  
255 |     return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};  
    |           ^~~~~~  
129 lines more ...  
2 errors generated.
```

Problems with `static_assert()`

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        static_assert(UnitCompatibleWith<ToU, unit, quantity_spec>,
            std::format("Unit '{}' is incompatible for '{}'", unit_symbol(ToU{}), unit_symbol(unit)));
        static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,
            std::format("Conversion from '{}' to '{}' is not value-preserving for '{}' representation type",
                unit_symbol(unit), unit_symbol(ToU{}), display_string_of(dealias(^rep))));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```


Problems with `static_assert()`

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        static_assert(UnitCompatibleWith<ToU, unit, quantity_spec>,
            std::format("Unit '{}' is incompatible for '{}'", unit_symbol(ToU{}), unit_symbol(unit)));
        static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,
            std::format("Conversion from '{}' to '{}' is not value-preserving for '{}' representation type",
                unit_symbol(unit), unit_symbol(ToU{}), display_string_of(dealias(^rep))));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

Even when the `static_assert` fails, the compiler still tries to determine the return type of the function.

Problems with `static_assert()`

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (requires { q.in(u); })
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

Problems with static_assert()

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (requires { q.in(u); })
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

error: static assertion failed: Conversion from 'm' to 'km' is not value-preserving for 'int' representation type

```
256 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,
      |                   ^~~~~~
note: in instantiation of function template specialization 'mp_units::quantity<mp_units::si::metre{{{}}}, int>::
      |       in<mp_units::si::kilo_<mp_units::si::metre>>' requested here
10  |     quantity q3 = q.in(km);
      |                   ^
note: because 'ValuePreservingTo<rep, rep, unit, mp_units::si::kilo_<mp_units::si::metre>{}>' evaluated to false
256 |     static_assert(ValuePreservingTo<rep, rep, unit, ToU{}>,
      |                   ^
note: because 'int' does not satisfy 'IsFloatingPoint'
      |                   (IsFloatingPoint<ToRep> || (!IsFloatingPoint<std::remove_cvref_t<FromRep>> &&
70  |                   ^
note: because 'treat_as_floating_point<int>' evaluated to false
65  |     concept IsFloatingPoint = treat_as_floating_point<T>;
      |                               ^
...
```

Problems with `static_assert()`

<> Code

Issues 9

Pull requests 4

Actions

Projects

Security

Insights

Do not use `static_assert()` to prevent template instantiation errors #77

Open

EditNew issue

mpusz opened on Apr 20, 2023

Using `static_assert()` allows great error messages but is a really bad solution for verification of a template instantiation. It does not SFINAE and is harmless to overload resolution.

For example, I just obtained the following error message:

```
[build] /home/mpusz/.conan2/p/wg21-711143a21ed44/p/include/linear_algebra/op_traits_multiplication.hpp: In instantiation of 'constexpr auto op_traits_multiplication::multiply(const T1&, const T2&) const [with T1 = int; T2 = int]: required from substitution of 'template<class _Tp, class _Up> constexpr auto op_traits_multiplication::multiply(const _Tp&, const _Up&) const [with _Tp = int; _Up = int]: required from substitution of 'template<class _Fn, class ... _Args> static constexpr auto op_traits_multiplication::multiply(const _Fn&, const _Args&) const [with _Fn = struct std::__result_of_impl<false, false, std::multiplies<int, int>>; _Args = {int, int}]: recursively required by substitution of 'template<class _Result, class ... _Args> static constexpr auto op_traits_multiplication::multiply(const _Result&, const _Args&) const [with _Result = struct std::__result_of_impl<false, false, std::multiplies<int, int>>; _Args = {int, int}]: required from 'struct std::is_invocable_v<std::multiplies<void>, std::expression>': required from 'constexpr const bool std::is_invocable_v<std::multiplies<void>, std::expression>': required by substitution of 'template<class Q> requires (Quantity<Q>) && (Q) op_traits_multiplication::operator*(const Q&) const [with Q = int]: required from here'
[build] /home/mpusz/.conan2/p/wg21-711143a21ed44/p/include/linear_algebra/op_traits_multiplication.hpp:171:61: error: static_assert((C1 == R2 || C1 == std::dynamic_extent || R2 == std::dynamic_extent), "multiplication engine traits")
[build] | ~~~~~~
```

After commenting out the `static_assert()` in `op_traits_multiplication.hpp`, my code compiled fine because a better overload was found.

Use concepts instead to prevent instantiations of types like `multiplication_engine_traits`.

Assignees

No one assigned

Labels

No labels

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Open in Workspace

No branches or pull requests

Notifications

CustomizeUnsubscribe

Problems with `static_assert()`

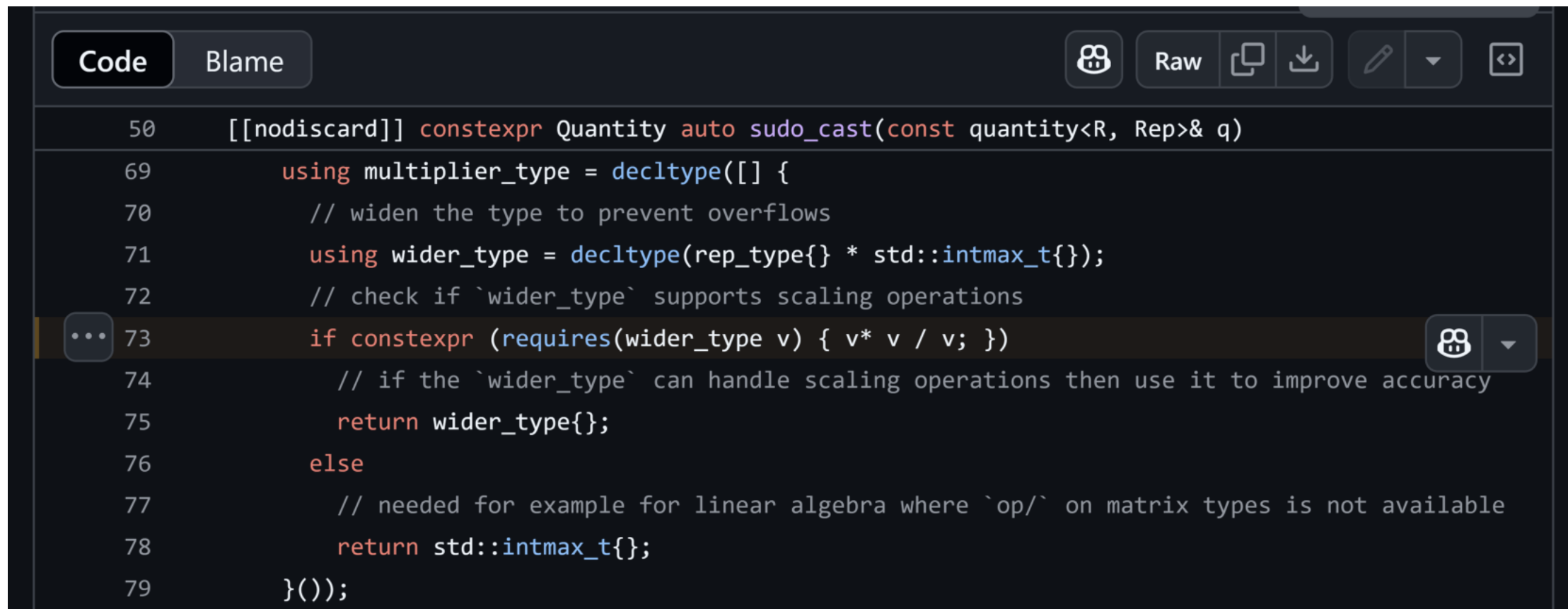
```
template<class COTR, class ET1, class ET2>
struct multiplication_engine_traits
{
    private:
        using element_type_1 = typename ET1::element_type;
        using element_type_2 = typename ET2::element_type;

        static constexpr size_t      R1 = engine_extents_helper<ET1>::rows();
        static constexpr size_t      C1 = engine_extents_helper<ET1>::columns();
        static constexpr size_t      R2 = engine_extents_helper<ET2>::rows();
        static constexpr size_t      C2 = engine_extents_helper<ET2>::columns();

        //- Validate the extents.
        //-
        static_assert((C1 == R2 || C1 == std::dynamic_extent || R2 == std::dynamic_extent),
                      "mis-matched/invalid number of rows and columns for multiplication");

        // ...
};
```

Problems with `static_assert()`



The screenshot shows a code editor with a dark theme. At the top, there are tabs for 'Code' and 'Blame', and icons for GitHub, Raw, Copy, Download, Edit, and a dropdown menu. The code is as follows:

```
50 [[nodiscard]] constexpr Quantity auto sudo_cast(const quantity<R, Rep>& q)
69     using multiplier_type = decltype([] {
70         // widen the type to prevent overflows
71         using wider_type = decltype(rep_type{} * std::intmax_t{});
72         // check if `wider_type` supports scaling operations
73         if constexpr (requires(wider_type v) { v* v / v; })
74             // if the `wider_type` can handle scaling operations then use it to improve accuracy
75             return wider_type{};
76         else
77             // needed for example for linear algebra where `op/` on matrix types is not available
78             return std::intmax_t{};
79     }());
```

Line 73 is highlighted with a dark background. To the left of line 73 is a button with three dots. To the right of line 73 is a GitHub icon and a dropdown arrow.

Problems with static_assert()

```
template<template<auto, typename> typename Q>
concept invalid_unit_conversion = requires {
    requires !requires { Q<isq::length[m], int>(2000 * m).in(km); }; // truncating conversion
    requires !requires { Q<isq::length[m], int>(2 * m).in(s); }; // unit of a different quantity & dimension
    requires !requires { Q<isq::frequency[Hz], int>(60 * Hz).in(Bq); }; // unit of a different kind (same dimension)
    requires !requires { Q<isq::length[m], int>(2 * m).force_in(s); }; // unit of a different quantity & dimension
    requires !requires {
        Q<isq::frequency[Hz], int>(60 * Hz).force_in(Bq);
    }; // unit of a different kind (same dimension)
};
static_assert(invalid_unit_conversion<quantity>);
```

Problems with `static_assert()`

```
template<template<auto, typename> typename Q>
concept invalid_unit_conversion = requires {
    requires !requires { Q<isq::length[m], int>(2000 * m).in(km); }; // truncating conversion
    requires !requires { Q<isq::length[m], int>(2 * m).in(s); }; // unit of a different quantity & dimension
    requires !requires { Q<isq::frequency[Hz], int>(60 * Hz).in(Bq); }; // unit of a different kind (same dimension)
    requires !requires { Q<isq::length[m], int>(2 * m).force_in(s); }; // unit of a different quantity & dimension
    requires !requires {
        Q<isq::frequency[Hz], int>(60 * Hz).force_in(Bq);
    }; // unit of a different kind (same dimension)
};
static_assert(invalid_unit_conversion<quantity>);
```

Usage of `static_assert` prevents unit testing of negative scenarios.

Problems with `static_assert()`

Introduction of `requires`-expressions in C++20 made it so easy to enable user-friendly development of smarter, safer, faster libraries.

Problems with `static_assert()`

Introduction of `requires`-expressions in C++20 made it so easy to enable user-friendly development of smarter, safer, faster libraries.

`static_assert` is not SFINAE-friendly. Prefer the usage of constraints and concepts to diagnose errors.

P2758 Emitting messages at compile time

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        if constexpr (!UnitCompatibleWith<ToU, unit, quantity_spec>)
            std::constexpr_error_str("invalid-unit-conversion",
                                     std::format("Unit '{}' is incompatible for '{}'",
                                                  unit_symbol(ToU{}), unit_symbol(unit)));
        if constexpr (!ValuePreservingTo<rep, rep, unit, ToU{}>)
            std::constexpr_error_str("invalid-unit-conversion",
                                     std::format("Conversion from '{}' to '{}' is not value-preserving "
                                                  "for '{}' representation type",
                                                  unit_symbol(unit), unit_symbol(ToU{}),
                                                  display_string_of(dealias(^rep))));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

Future extensions to P2758 with constexpr std::format

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        if constexpr (!UnitCompatibleWith<ToU, unit, quantity_spec>)
            std::constexpr_error("invalid-unit-conversion", "Unit '{}' is incompatible for '{}'",
                                unit_symbol(ToU{}), unit_symbol(unit));
        if constexpr (!ValuePreservingTo<rep, rep, unit, ToU{}>)
            std::constexpr_error("invalid-unit-conversion",
                                "Conversion from '{}' to '{}' is not value-preserving for '{}' representation type",
                                unit_symbol(unit), unit_symbol(ToU{}), display_string_of(dealias(^rep)));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

P2758 Emitting messages at compile time

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (requires { q.in(u); })
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

P2758 Emitting messages at compile time

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (requires { q.in(u); })
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

Not SFINAE-friendly 🙄

C++ exceptions

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
    [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
    {
        if constexpr (!UnitCompatibleWith<ToU, unit, quantity_spec>)
            throw invalid_unit_conversion(std::format("Unit '{}' is incompatible for '{}'",
                                                    unit_symbol(ToU{}), unit_symbol(unit)));
        if constexpr (!ValuePreservingTo<rep, rep, unit, ToU{}>)
            throw invalid_unit_conversion(std::format("Conversion from '{}' to '{}' is not value-preserving "
                                                    "for '{}' representation type",
                                                    unit_symbol(unit), unit_symbol(ToU{}),
                                                    display_string_of(dealias(^rep))));
        return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
    }
    // ...
};
```

C++ exceptions

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: no matching constructor for initialization of 'quantity<detail::make_reference(quantity_spec, kilo_<metre>{}), int>'  
281 |         return quantity<detail::make_reference(quantity_spec, ToU{}), Rep>{*this};  
    |         ^~~~~~
```

132 lines more...

1 error generated.

C++ exceptions

```
template<typename Rep, Unit FromU, Unit ToU, QuantitySpec QS>
constexpr bool safely_convertible(FromU, ToU, QS)
{
    if constexpr (!UnitCompatibleWith<ToU, FromU{}, QS{}>)
        throw invalid_unit_conversion(std::format("Unit '{}' is incompatible for '{}'",
                                                    unit_symbol(ToU{}), unit_symbol(FromU{})));
    if constexpr (!ValuePreservingTo<Rep, Rep, FromU{}, ToU{}>)
        throw invalid_unit_conversion(std::format("Conversion from '{}' to '{}' is not value-preserving "
                                                    "for '{}' representation type",
                                                    unit_symbol(FromU{}), unit_symbol(ToU{}),
                                                    display_string_of(dealias(^Rep))));

    return true;
}
```

C++ exceptions

```
template<typename Rep, Unit FromU, Unit ToU, QuantitySpec QS>
constexpr bool safely_convertible(FromU, ToU, QS)
{
    if constexpr (!UnitCompatibleWith<ToU, FromU{}, QS{}>)
        throw invalid_unit_conversion(std::format("Unit '{}' is incompatible for '{}'",
                                                    unit_symbol(ToU{}), unit_symbol(FromU{})));
    if constexpr (!ValuePreservingTo<Rep, Rep, FromU{}, ToU{}>)
        throw invalid_unit_conversion(std::format("Conversion from '{}' to '{}' is not value-preserving "
                                                    "for '{}' representation type",
                                                    unit_symbol(FromU{}), unit_symbol(ToU{}),
                                                    display_string_of(dealias(^Rep))));

    return true;
}
```

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity {
public:
    template<Unit ToU>
        requires (safely_convertible<rep>(unit, ToU{}, quantity_spec))
        [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const
        {
            return quantity<make_reference(quantity_spec, ToU{}), Rep>{*this};
        }
    // ...
};
```

C++ exceptions

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

error: substitution into constraint expression resulted in a non-constant expression

```
265 |     requires(detail::can_safely_convert<rep>(unit, ToU{}), quantity_spec))  
    |             ^~~~~~
```

note: while checking constraint satisfaction for template 'in<mp_units::si::metre{{{}}}, int>' required here

```
22 |     constexpr quantity q3 = q.in(km);  
    |                             ^~
```

note: in instantiation of function template specialization 'mp_units::quantity<mp_units::si::metre{{{}}}, int>::in<mp_units::si::kilo<mp_units::si::metre>>' requested here

note: subexpression not valid in a constant expression

```
153 |         throw invalid_unit_conversion(std::format("Conversion from '{}'" to '{}'" is not value-preserving "  
    |             ^~~~~~
```

```
154 |                                     "for '{}'" representation type",  
    |                                     ~~~~~~
```

```
155 |                                     unit_symbol(FromU{}), unit_symbol(ToU{}),  
    |                                     ~~~~~~
```

```
156 |                                     display_string_of(dealias(^Rep))));  
    |                                     ~~~~~~
```

note: in call to 'can_safely_convert<int, mp_units::si::metre, mp_units::si::kilo<mp_units::si::metre>, mp_units::kind_of<mp_units::isq::length>({{}}), {{{}}, {{{}}}, {{{}}})'

```
265 |     requires(detail::can_safely_convert<rep>(unit, ToU{}), quantity_spec))  
    |             ^~~~~~
```

error: no matching member function for call to 'in'

```
22 |     constexpr quantity q3 = q.in(km);  
    |                             ~~~~
```

note: candidate template ignored: couldn't infer template argument 'ToRep'

```
293 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                             ^
```

note: candidate template ignored: failed template argument deduction

```
266 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                             ^
```

note: candidate function template not viable: requires 0 arguments, but 1 was provided

```
286 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in() const  
    |                             ^
```

2 errors generated.

P3068 Allowing exception throwing in constant-evaluation (C++26)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

error: no matching member function for call to 'in'

```
10 |     quantity q3 = q.in(km);  
    |                   ~~~^~
```

note: candidate template ignored: constraints not satisfied [with ToU = kilo_<decltype(metre{{{}}})>]

```
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                                     ^
```

note: because 'safely_convertible<rep>(unit, ToU{}, quantity_spec)' throws an exception

```
243 |     requires (safely_convertible<rep>(unit, ToU{}, quantity_spec))  
    |               ^
```

note: unhandled exception: Conversion from 'm' to 'km' is not value-preserving for 'int' representation type

```
153 |     throw invalid_unit_conversion(std::format("Conversion from '{}'" to '{}'" is not value-preserving "  
    |       ^
```

1 error generated.

P3068 Allowing exception throwing in constant-evaluation (C++26)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: no matching member function for call to 'in'  
10 |     quantity q3 = q.in(km);  
   |                   ~~~~  
note: candidate template ignored: constraints not satisfied [with ToU = kilo_<decltype(metre{{{}}})>]  
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
   |                       ^  
note: because 'safely_convertible<rep>(unit, ToU{}, quantity_spec)' throws an exception  
243 |     requires (safely_convertible<rep>(unit, ToU{}, quantity_spec))  
   |               ^  
note: unhandled exception: Conversion from 'm' to 'km' is not value-preserving for 'int' representation type  
153 |     throw invalid_unit_conversion(std::format("Conversion from '{}'' to '{}'' is not value-preserving "  
   |     ^  
1 error generated.
```



P3068 Allowing exception throwing in constant-evaluation (C++26)

```
quantity q = 42 * m;  
quantity q3 = q.in(km);
```

```
error: no matching member function for call to 'in'  
10 |     quantity q3 = q.in(km);  
    |                   ~~~~  
note: candidate template ignored: constraints not satisfied [with ToU = kilo_<decltype(metre{{{}}})>]  
244 |     [[nodiscard]] constexpr QuantityOf<quantity_spec> auto in(ToU) const  
    |                   ^  
note: because 'safely_convertible<rep>(unit, ToU{}, quantity_spec)' throws an exception  
243 |     requires (safely_convertible<rep>(unit, ToU{}, quantity_spec))  
    |                   ^  
note: unhandled exception: Conversion from 'm' to 'km' is not value-preserving for 'int' representation type  
153 |     throw invalid_unit_conversion(std::format("Conversion from '{}' to '{}' is not value-preserving "  
    |     ^  
1 error generated.
```



Does not provide exact information about which exact check failed and why, but do we really need that?

P3068 Allowing exception throwing in constant-evaluation (C++26)

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (requires { q.in(u); })
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

P3068 Allowing exception throwing in constant-evaluation (C++26)

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (requires { q.in(u); })
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

Not SFINAE-friendly 😬

P3068 Allowing exception throwing in constant-evaluation (C++26)

```
constexpr bool not_throwing(std::predicate auto func)
{
    try { return func(); }
    catch(...) { return false; }
}
```

P3068 Allowing exception throwing in constant-evaluation (C++26)

```
constexpr bool not_throwing(std::predicate auto func)
{
    try { return func(); }
    catch(...) { return false; }
}
```

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (not_throwing([]{ return safely_convertible<typename Q::rep>(Q::unit, U{}, Q::quantity_spec); }))
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

P3068 Allowing exception throwing in constant-evaluation (C++26)

```
constexpr bool not_throwing(std::predicate auto func)
{
    try { return func(); }
    catch(...) { return false; }
}
```

```
template<Quantity Q, Unit U>
constexpr Quantity auto try_convert_to(Q q, U u)
{
    if constexpr (not_throwing([]{ return safely_convertible<typename Q::rep>(Q::unit, U{}, Q::quantity_spec); })))
        return q.in(u);
    else
        return q;
}
```

```
quantity q = 42 * m;
quantity q4 = try_convert_to(q, km);
```

Works, but far from being user friendly.

Extending P3068 to make constexpr exceptions SFINAE-friendly

- [temp.constr.op]

- ⁵ [Note 2: A logical negation expression (7.6.2.2) is an atomic constraint; the negation operator is not treated as a logical operation on constraints. As a result, distinct negation *constraint-expressions* that are equivalent under 13.7.7.2 do not subsume one another under 13.5.5. Furthermore, if substitution to determine whether an atomic constraint is satisfied (13.5.2.3) encounters a substitution failure, the constraint is not satisfied, regardless of the presence of a negation operator.

Extending P3068 to make constexpr exceptions SFINAE-friendly

- [temp.constr.op]

- ⁵ [Note 2: A logical negation expression (7.6.2.2) is an atomic constraint; the negation operator is not treated as a logical operation on constraints. As a result, distinct negation *constraint-expressions* that are equivalent under 13.7.7.2 do not subsume one another under 13.5.5. Furthermore, if substitution to determine whether an atomic constraint is satisfied (13.5.2.3) encounters a substitution failure, the constraint is not satisfied, regardless of the presence of a negation operator.

This should be extended with "or results with an exception being thrown".

P3378 constexpr exception types (C++26)

```
struct invalid_unit_conversion : std::invalid_argument {  
    constexpr explicit invalid_unit_conversion(const std::string& msg):  
        std::invalid_argument(msg)  
    {  
    }  
};
```

IMPROVING SAFETY

Preventing truncation of data

 Improving Our Safety With a Quantities and Units Library

think-cell

Preventing truncation of data

```
quantity q1 = 5 * m;  
std::cout << q1.in(km) << '\n';  
quantity<si::kilo<si::metre>, int> q2 = q1; // Compile-time error
```

- Floating-point representation type **is** considered **value-preserving**

```
quantity q1 = 5. * m; // source quantity uses 'double' as a representation type  
std::cout << q1.in(km) << '\n';  
quantity<si::kilo<si::metre>> q2 = q1;
```

```
quantity q1 = 5 * m; // source quantity uses 'int' as a representation type  
std::cout << q1.in<double>(km) << '\n';  
quantity<si::kilo<si::metre>> q2 = q1; // 'double' by default
```

 C++ on Sea 2024: Improving our safety with a quantities and units library 67

Mateusz Pusz

cpponseas.uk 45:37 / 1:31:14 2024

Improving Our Safety With a Modern C++ Quantities and Units Library - Mateusz Pusz - C++ on Sea 2024

Preventing truncation of data

```
template<typename FromRep, typename ToRep, auto FromUnit, auto ToUnit>
concept ValuePreservingTo =
    Unit<decltype(FromUnit)> && Unit<decltype(ToUnit)> &&
    std::assignable_from<ToRep&, FromRep> &&
    (treat_as_floating_point<ToRep> ||
     (!treat_as_floating_point<std::remove_cvref_t<FromRep>> &&
      integral_conversion_factor(FromUnit, ToUnit) &&
      !overflows_non_zero_values<ToRep>(FromUnit, ToUnit)));
```

Klaus Iglberger: The Real Problem of C++

C++'s Superpower: Abstractions

```
class Second
{
public:
    constexpr Second() = default;

    template< typename U > requires std::same_as<U,double>
    explicit constexpr Second( U value ) : value_{value} { /*possible checks*/ }

    template< typename U > requires std::same_as<U,double>
    constexpr Second& operator=( U value ) { /*Possible checks*/ value_ = value; return *this; }

    [[nodiscard]] constexpr double value() const { return value_; }

    // ...

private:
    double value_{};
};
```



This prevents any kind of conversion!

Klaus Iglberger: The Real Problem of C++

C++'s Superpower: Abstractions

```
class Second
{
public:
    constexpr Second() = default;

    template< typename U > requires no_narrowing_conversion<U, double>
    explicit constexpr Second( U value ) : value_{value} { /*possible checks*/ }

    template< typename U > requires no_narrowing_conversion<U, double>
    constexpr Second& operator=( U value ) { /*Possible checks*/ value_ = value; return *this; }

    [[nodiscard]] constexpr double value() const { return value_; }

    // ...

private:
    double value_{};
};
```



YOU control the conversions!

std::simd (P1928)

(7.1) **29.10 Data-parallel types** [simd]

(7.1.1) **29.10.1 General** [simd.general]

- 8 The conversion from an arithmetic type U to a vectorizable type T is *value-preserving* if all possible values of U can be represented with type T .

P2509 A proposal for a type trait to detect value-preserving conversions (Abandoned)

```
template<typename FromRep, typename ToRep, auto FromUnit, auto ToUnit>
concept ValuePreservingTo =
    Unit<decltype(FromUnit)> && Unit<decltype(ToUnit)> &&
    std::assignable_from<ToRep&, FromRep> &&
    std::is_value_preserving_convertible_v<FromRep, ToRep> &&
    (treat_as_floating_point<ToRep> ||
     (!treat_as_floating_point<std::remove_cvref_t<FromRep>> &&
      integral_conversion_factor(FromUnit, ToUnit) &&
      !overflows_non_zero_values<ToRep>(FromUnit, ToUnit)));
```

A NEED FOR ADDITIONAL TEXT UTILITIES

A simple "legacy" C++ program

```
std::cout << "What is your name?\n";  
std::string name;  
std::cin >> name;  
std::cout << "Hello " << name << "!\n";
```

A Modern C++ program

```
std::println("What is your name?");  
std::string name;  
std::cin >> name;  
std::println("Hello {}!", name);
```


A Modern C++ program

```
std::println("What is your name?");  
std::string name;  
std::cin >> name;  
std::println("Hello {}!", name);
```



NOT a Modern C++ program

```
std::println("What is your name?");  
std::string name;  
std::cin >> name;  
std::println("Hello {}!", name);
```

NOT a Modern C++ program

```
std::println("What is your name?");  
std::string name;  
std::cin >> name;  
std::println("Hello {}", name);
```

- I/O streams
- Default construction followed by assignment
- No error handling

P1729 Text Parsing

WITH EXCEPTIONS

```
auto [name] = scn::prompt<std::string>("What is your name?\n", "{}")->values();  
std::println("Hello {}!", name);
```

P1729 Text Parsing

WITH EXCEPTIONS

```
auto [name] = scn::prompt<std::string>("What is your name?\n", "{}")->values();  
std::println("Hello {}!", name);
```

NO EXCEPTIONS

```
if (auto result = scn::prompt<std::string>("What is your name?\n", "{}")) {  
    auto [name] = result->values();  
    std::println("Hello {}!", name);  
}  
else  
    std::println(stderr, "Error: {}", result.error().msg());
```

Internationalization

Isn't it embarrassing that in 2025 we do not have a standardized tools for Unicode or text internationalization?

IMPROVING DISTRIBUTION AND CONSUMPTION OF LIBRARIES

P2673 Common Description Format for C++ Libraries and Packages

BEFORE

- mp-unitsConfig.cmake

```
# mp-unitsConfig.cmake
include(CMakeFindDependencyMacro)

if(NOT MP_UNITS_API_FREESTANDING AND NOT MP_UNITS_API_STD_FORMAT)
    find_dependency(fmt)
endif()

if(MP_UNITS_API_CONTRACTS STREQUAL "GSL-LITE")
    find_dependency(gsl-lite)
elseif(MP_UNITS_API_CONTRACTS STREQUAL "MS-GSL")
    find_dependency(Microsoft.GSL)
endif()

include("${CMAKE_CURRENT_LIST_DIR}/mp-unitsTargets.cmake")
```

- CMakeLists.txt

```
# ...
configure_file("mp-unitsConfig.cmake" "." COPYONLY)
include(CMakePackageConfigHelpers)
write_basic_package_version_file(mp-unitsConfigVersion.cmake
                                COMPATIBILITY SameMajorVersion)

install(EXPORT mp-unitsTargets
        DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/mp-units
        NAMESPACE mp-units::)

install(FILES mp-unitsConfig.cmake
        ${CMAKE_CURRENT_BINARY_DIR}/mp-unitsConfigVersion.cmake
        DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/mp-units)
```


P2673 Common Description Format for C++ Libraries and Packages

BEFORE

- mp-unitsConfig.cmake

```
# mp-unitsConfig.cmake
include(CMakeFindDependencyMacro)

if(NOT MP_UNITS_API_FREESTANDING AND NOT MP_UNITS_API_STD_FORMAT)
    find_dependency(fmt)
endif()

if(MP_UNITS_API_CONTRACTS STREQUAL "GSL-LITE")
    find_dependency(gsl-lite)
elseif(MP_UNITS_API_CONTRACTS STREQUAL "MS-GSL")
    find_dependency(Microsoft.GSL)
endif()

include("${CMAKE_CURRENT_LIST_DIR}/mp-unitsTargets.cmake")
```

- CMakeLists.txt

```
# ...
configure_file("mp-unitsConfig.cmake" "." COPYONLY)
include(CMakePackageConfigHelpers)
write_basic_package_version_file(mp-unitsConfigVersion.cmake
                                COMPATIBILITY SameMajorVersion)

install(EXPORT mp-unitsTargets
        DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/mp-units
        NAMESPACE mp-units::)

install(FILES mp-unitsConfig.cmake
        ${CMAKE_CURRENT_BINARY_DIR}/mp-unitsConfigVersion.cmake
        DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/mp-units)
```

AFTER

- CMakeLists.txt

```
# ...
install(PACKAGE_INFO mp-units
        EXPORT mp-unitsTargets)
```

P2673 Common Description Format for C++ Libraries and Packages

BEFORE

```
class MPUnitsConan(ConanFile):
    # ...
    def package_info(self):
        compiler = self.settings.compiler
        # TODO remove the branch when Conan will learn to handle C++ modules
        if self.options.cxx_modules:
            # CMakeDeps does not generate C++ modules definitions for now
            # Skip the Conan-generated files and use the mp-unitsConfig.cmake bundled with mp-units
            self.cpp_info.set_property("cmake_find_mode", "none")
            self.cpp_info.builddirs = ["."]
        else:
            # handle contracts
            if self.options.contracts == "none":
                self.cpp_info.components["core"].defines.append(
                    "MP_UNITS_API_CONTRACTS=0"
                )
            elif self.options.contracts == "gsl-lite":
                self.cpp_info.components["core"].requires.append("gsl-lite::gsl-lite")
                self.cpp_info.components["core"].defines.append(
                    "MP_UNITS_API_CONTRACTS=2"
                )
            elif self.options.contracts == "ms-gsl":
                self.cpp_info.components["core"].requires.append("ms-gsl::ms-gsl")
                self.cpp_info.components["core"].defines.append(
                    "MP_UNITS_API_CONTRACTS=3"
                )

            # handle API options
            self.cpp_info.components["core"].defines.append(
                "MP_UNITS_API_NO_CRTP=" + str(int(self.options.no_crtip == True))
            )
            self.cpp_info.components["core"].defines.append(
                "MP_UNITS_API_STD_FORMAT=" + str(int(self.options.std_format == True))
            )
            if not self.options.std_format:
                self.cpp_info.components["core"].requires.append("fmt::fmt")

            # handle hosted configuration
            if not self.options.freestanding:
                self.cpp_info.components["core"].defines.append("MP_UNITS_HOSTED=1")

            # handle import std
            if self.options.import_std:
                self.cpp_info.components["core"].defines.append("MP_UNITS_IMPORT_STD")
                if compiler == "clang" and Version(compiler.version) < 19:
                    self.cpp_info.components["core"].cxxflags.append(
                        "-Wno-deprecated-declarations"
                    )

            if compiler == "msvc":
                self.cpp_info.components["core"].cxxflags.append("/utf-8")

            self.cpp_info.components["systems"].requires = ["core"]
```

AFTER

```
class MPUnitsConan(ConanFile):
    # ...
    def package_info(self):
        from conan.cps import CPS
        self.cpp_info = CPS.load("cps/mp-units.cps").to_conan()
```

P2673 Common Description Format for C++ Libraries and Packages

BEFORE

```
class MPUnitsConan(ConanFile):
    # ...
    def package_info(self):
        compiler = self.settings.compiler
        # TODO remove the branch when Conan will learn to handle C++ modules
        if self.options.cxx_modules:
            # CMakeDeps does not generate C++ modules definitions for now
            # Skip the Conan-generated files and use the mp-unitsConfig.cmake bundled with mp-units
            self.cpp_info.set_property("cmake_find_mode", "none")
            self.cpp_info.builddirs = ["."]
        else:
            # handle contracts
            if self.options.contracts == "none":
                self.cpp_info.components["core"].defines.append(
                    "MP_UNITS_API_CONTRACTS=0"
                )
            elif self.options.contracts == "gsl-lite":
                self.cpp_info.components["core"].requires.append("gsl-lite::gsl-lite")
                self.cpp_info.components["core"].defines.append(
                    "MP_UNITS_API_CONTRACTS=2"
                )
            elif self.options.contracts == "ms-gsl":
                self.cpp_info.components["core"].requires.append("ms-gsl::ms-gsl")
                self.cpp_info.components["core"].defines.append(
                    "MP_UNITS_API_CONTRACTS=3"
                )

            # handle API options
            self.cpp_info.components["core"].defines.append(
                "MP_UNITS_API_NO_CRTP=" + str(int(self.options.no_crt_p == True))
            )
            self.cpp_info.components["core"].defines.append(
                "MP_UNITS_API_STD_FORMAT=" + str(int(self.options.std_format == True))
            )
            if not self.options.std_format:
                self.cpp_info.components["core"].requires.append("fmt::fmt")

            # handle hosted configuration
            if not self.options.freestanding:
                self.cpp_info.components["core"].defines.append("MP_UNITS_HOSTED=1")

            # handle import std
            if self.options.import_std:
                self.cpp_info.components["core"].defines.append("MP_UNITS_IMPORT_STD")
                if compiler == "clang" and Version(compiler.version) < 19:
                    self.cpp_info.components["core"].cxxflags.append(
                        "-Wno-deprecated-declarations"
                    )

            if compiler == "msvc":
                self.cpp_info.components["core"].cxxflags.append("/utf-8")

            self.cpp_info.components["systems"].requires = ["core"]
```

FUTURE

```
class MPUnitsConan(ConanFile):
    # ...
```

P2673 Common Description Format for C++ Libraries and Packages

```
cmake --install . --config Release
```

BEFORE

```
$ tree lib
.
├── include
│   └── mp-units
│       └── ...
└── lib
    ├── cmake
    │   └── mp-units
    │       ├── mp-unitsConfig.cmake
    │       ├── mp-unitsConfigVersion.cmake
    │       └── mp-unitsTargets.cmake
```

P2673 Common Description Format for C++ Libraries and Packages

```
cmake --install . --config Release
```

BEFORE

```
$ tree lib
.
├── include
│   └── mp-units
│       └── ...
└── lib
    ├── cmake
    │   └── mp-units
    │       ├── mp-unitsConfig.cmake
    │       ├── mp-unitsConfigVersion.cmake
    │       └── mp-unitsTargets.cmake
```

AFTER

```
$ tree lib
.
├── include
│   └── mp-units
│       └── ...
└── lib
    ├── cps
    │   ├── mp-units
    │   └── mp-units.cps
```

P2673 Common Description Format for C++ Libraries and Packages

```
{
  "components" :
  {
    "core" :
    {
      "compile_definitions" :
      {
        "*" :
        {
          "MP_UNITS_API_NO_CRTP" : "1",
          "MP_UNITS_API_STD_FORMAT" : "0",
          "MP_UNITS_HOSTED" : "1"
        }
      },
      "compile_features" : [ "c++26" ],
      "compile_flags" : [ "-Wno-unused-result" ],
      "requires" : [ "fmt:fmt", ":mp-units-contracts" ],
      "type" : "interface"
    },
    "mp-units" :
    {
      "compile_features" : [ "c++26" ],
      "requires" : [ ":core", ":systems" ],
      "type" : "interface"
    }
  },
  "mp-units-contracts" :
  {
    "compile_definitions" :
    {
      "*" :
      {
        "MP_UNITS_API_CONTRACTS" : "0"
      }
    },
    "type" : "interface"
  },
  "systems" :
  {
    "compile_features" : [ "c++26" ],
    "requires" : [ ":core" ],
    "type" : "interface"
  }
},
"cps_path" : "@prefix@/lib/cps/mp-units",
"cps_version" : "0.12.0",
"name" : "mp-units",
"requires" :
{
  "fmt" : null
}
}
```

```
"mp-units-contracts" :
{
  "compile_definitions" :
  {
    "*" :
    {
      "MP_UNITS_API_CONTRACTS" : "0"
    }
  },
  "type" : "interface"
},
"systems" :
{
  "compile_features" : [ "c++26" ],
  "requires" : [ ":core" ],
  "type" : "interface"
}
},
"cps_path" : "@prefix@/lib/cps/mp-units",
"cps_version" : "0.12.0",
"name" : "mp-units",
"requires" :
{
  "fmt" : null
}
}
```

SUMMARY

C++ is awesome!

C++ is probably the best programming language to implement fast generic libraries.

C++ is awesome!

C++ is probably the best programming language to implement fast generic libraries.

- **Zero-cost abstractions** thanks to C++ templates
 - runtime performance
 - memory usage

C++ is awesome!

C++ is probably the best programming language to implement fast generic libraries.

- **Zero-cost abstractions** thanks to C++ templates
 - runtime performance
 - memory usage
- Constraints and concepts improve **interfaces**

C++ is awesome!

C++ is probably the best programming language to implement fast generic libraries.

- Zero-cost abstractions thanks to C++ templates
 - runtime performance
 - memory usage
- Constraints and concepts improve interfaces
- Contracts will improve safety and interfaces

C++ is awesome!

C++ is probably the best programming language to implement fast generic libraries.

- Zero-cost abstractions thanks to C++ templates
 - runtime performance
 - memory usage
- Constraints and concepts improve interfaces
- Contracts will improve safety and interfaces
- Reflection will change the C++ world

Please help!

Some features are still missing to make C++ even better.

Please help!

Some features are still missing to make C++ even better.

If you are an expert or just care about a specific subject, please help!



CAUTION
Programming
is addictive
(and too much fun)